



INDUSTRIAL ROBOTICS

COURSEWORK ASSIGNMENT – 1ST SIT

Design, Simulation, and Kinematic Analysis of an
Automated Picking and Sorting Workcell for Phones-R-Us



ROBOTIC
AUTOMATION



PICK & SORT
SYSTEM



SIMULATION
(COPPELIASIM)



KINEMATIC
ANALYSIS (MATLAB)



SAFETY & RISK
ASSESSMENT



PERFORMANCE
EVALUATION

ASSESSMENT REPORT

Name:	Muhammad Faizan Ahmed
Student ID:	25937922
Module:	6E7V0024 Industrial Robotics
Institution:	Manchester Metropolitan University
Academic Year:	2025–2026

ABSTRACT

The increasing demand for productivity, consistency, and flexibility in manufacturing has encouraged industries to adopt robotic automation systems for repetitive handling and packaging tasks. This project presents the design, simulation, and analysis of an automated robotic workcell developed for the Phones-R-Us production scenario, where phone cases are transported, identified, sorted, and packaged automatically. The proposed system aims to improve throughput, reduce manual intervention, and support future production scalability while meeting the target requirement of processing one phone case every two seconds.

A collaborative Universal Robots UR20 manipulator was selected due to its six degrees of freedom, 20 kg payload capacity, 1.75 m reach, and suitability for industrial pick-and-place operations. The workcell was developed in CoppeliaSim and includes a conveyor system, colour-based vision inspection, proximity sensing, defect rejection handling, packaging boxes, and safety monitoring mechanisms. Lua scripts were implemented to manage robot motion, conveyor control, object detection, state transitions, collision avoidance, and operational sequencing.

MATLAB was used to perform a complete kinematic analysis of the UR20 robot using the imported URDF model. Forward kinematics, inverse kinematics, workspace analysis, joint-space validation, multiple inverse kinematic solutions, and trajectory feasibility studies were carried out for the pick and place operations. The results were further used to justify simulation layout decisions, waypoint placement, and robot motion planning.

Additional analyses including safety assessment, risk mitigation strategies, performance evaluation, and PESTLE considerations were conducted to assess industrial feasibility. The developed solution demonstrates a reliable and scalable robotic automation system capable of supporting future manufacturing expansion and improved operational efficiency.

Table of Contents

.....	1
1. INTERPRETATION OF CLIENT REQUIREMENTS.....	6
1.1. Overview of the Phones-R-Us Production Scenario	6
1.2. Production Targets and Performance Requirements	6
1.3. Operational Constraints.....	7
1.4. Motivation for Automation	8
2. Proposed Robotic Solution	9
2.1. Introduction to the Proposed System	9
2.2. Robot Payload Requirement Analysis.....	9
2.3. Comparative Evaluation of Universal Robots Manipulators.....	10
2.4. End Effector Selection	12
2.5. Sensor and Perception System	14
2.6. Safety System Design	15
2.7. Motion Planning and Workspace Design.....	16
2.8. Control Architecture and System Logic.....	17
3. Implementation and Simulation Results	20
3.1. Simulation Environment Setup	20
.....	20
3.2. System Implementation.....	20
3.3. Sorting Operation Results.....	21
3.4. User Interface and System Monitoring.....	23
4. MATLAB Kinematic Analysis	24
4.1. MATLAB Environment and Robot Model.....	24
4.2 Forward Kinematics Analysis	25
4.3 Inverse Kinematics Analysis.....	26
4.4 Workspace and Reachability Validation	28
4.5 Cartesian Motion Planning and Path Validation	30
5. Safety Assessment and Risk Mitigation.....	31
6. PESTLE Analysis.....	32
7. Performance Evaluation and Limitations	35

7.1. Cycle Time and Throughput Evaluation	35
7.2. System Limitations.....	35
7.3. Future Improvements	36
References	37
Appendix B	41
Appendix C.....	43
Appendix D	45
Appendix F	48
Appendix G	49
Appendix H	50

Table of Figures

Figure 1: Proposed UR20 robotic sorting and packing workcell implemented within the Coppeliasim environment.....	9
Figure 2: UR20 collaborative industrial robot selected for the proposed automation system ..	11
Figure 3: OnRobot VGC10 electrical vacuum gripper selected for the proposed robotic handling system	12
Figure 4: UR20 collaborative robot integrated with the OnRobot VGC10 vacuum gripper	13
Figure 5: Sensor placement and inspection arrangement within the proposed robotic workcell	14
Figure 6: Overall operational sequence implemented within the robotic sorting system	17
Figure 7: Finite state machine for sorting and box handling operation	18
Figure 8: Integrated control architecture of the proposed robotic sorting and packing system	19
Figure 9: Complete Coppeliasim implementation of the Phones-R-Us robotic workcell	20
Figure 10: Conveyor inspection and robotic pickup sequence within the sorting process	21
Figure 11: Validation of colour-based sorting and defective product rejection operations	22
Figure 12: Graphical user interface developed for simulation control, testing and live production monitoring within the Coppeliasim environment	23
Figure 13: Graphical user interface developed for robotic system operation and monitoring ..	23
Figure 14: Reconstructed UR20 robotic workcell environment in MATLAB using imported URDF and collision objects	24
Figure 15: MATLAB IK Designer environment showing validated UR20 industrial task poses..	25

Figure 16: Forward kinematic TCP positions generated for validated industrial task poses	26
Figure 17: Inverse kinematic validation results for industrial task poses	27
Figure 18: Validated UR20 inverse kinematic configurations for industrial sorting and handling operations	27
Figure 19: Three-dimensional UR20 workspace validation showing reachable operating volume and validated industrial task targets.....	28
Figure 20: Reachability validation results for major industrial task locations	28
Figure 21: Reachability validation analysis table	29
Figure 22: Reachability validation of major industrial handling poses within the UR20 workcell	29
Figure 23: Actual UR20 industrial motion paths implemented within the CoppeliaSim robotic sorting environment.....	30
Figure 24: Universal Robots e-Series technical specification comparison used during robot evaluation.....	38
Figure 25: Figure A2: Manufacturer technical specification sheet for the selected UR20 collaborative robot.....	39
Figure 26: Manufacturer technical specification sheet for the UR30 collaborative robot evaluated during selection.....	40
Figure 27: Automatic box filling and transportation operations implemented within the robot control system.....	41
Figure 28: Top-view arrangement of phone case placement positions generated by the robot control logic.....	42
Figure 29: Vision sensor colour classification and sorting logic.....	43
Figure 30: Automatic box handling and replacement sequence implemented within the robot control system.....	44
Figure 31: TCP coordinates generated using forward kinematic analysis for validated industrial task poses	47
Figure 32: Joint-space configurations for validated UR20 industrial task poses.....	48
Figure 33: Top-view workspace validation of industrial task locations within the UR20 workcell	49

1. INTERPRETATION OF CLIENT REQUIREMENTS

1.1. Overview of the Phones-R-Us Production Scenario

Phones-R-Us requires an automated manufacturing solution for sorting and packaging phone cases produced on a conveyor-based production line. The existing manual process involves repetitive handling, product inspection and sorting operations which may reduce production consistency, increase labour dependency and limit production throughput.

The developed solution focuses on automating the handling of coloured phone cases transported through the conveyor system. Once a product reaches the inspection region, sensors identify the incoming variant and the robotic manipulator transfers the unit to the corresponding sorting box. Defective products are automatically separated and placed into a dedicated rejection area.

The proposed system therefore performs the following industrial tasks:

- Continuous conveyor transport of phone cases
- Automatic colour classification
- Pick and place manipulation using an industrial robot
- Sorting into packaging locations
- Defect rejection handling
- Safety monitoring and controlled operation
- Repeatable production cycles

1.2. Production Targets and Performance Requirements

The client requirement specifies that the proposed automation system should achieve a production rate equivalent to processing approximately one phone case every two seconds. Therefore, the robotic workcell must operate continuously while maintaining repeatability, collision avoidance, and safe interaction conditions. Table 1 summarises the main production requirements.

Production Requirements

Parameter	Requirement
Product Type	mPhone 26 unit
Product Variants	Obsidian Black, Crimson Red, Quantum Blue
Defective Parts	Automatic rejection
Robot Task	Pick-and-place handling
Sorting Method	Sensor-based classification
Throughput Target	1 phone / 2 s
Operation Mode	Automated continuous cycle
Safety Requirement	Collaborative operation

Table 1: Production and operational requirements for the Phones-R-Us robotic sorting system

1.3. Operational Constraints

Several engineering and operational constraints influenced the development of the proposed robotic workcell.

Workspace Constraint

The robot must operate inside a fixed industrial cell where the conveyor, storage boxes, rejection area, sensors, and robot base occupy predefined positions. The available reach of the manipulator directly influences system layout.

Continuous Production Requirement

Phone cases continuously arrive through the conveyor system. The robot must therefore perform detection, gripping, transport, and placement operations without interrupting production flow.

Product Handling Constraint

Phone cases possess relatively small geometry and low mass. The gripping mechanism must therefore provide stable handling while avoiding surface damage or misalignment during placement.

Reachability Constraint

All required locations including the conveyor pick point, coloured boxes, and reject area must remain inside the robot workspace while maintaining acceptable joint motion.

Safety Constraint

The system is intended to operate in an industrial environment where collaborative interaction may occur. Safety measures including speed control, monitored zones, emergency stop logic, and safe approach paths become necessary.

Motion Constraint

Collision-free trajectories are required to prevent interference between:

- Robot links
- Conveyor
- Packaging boxes
- Sensors
- End effector paths

These constraints directly influenced robot selection and workcell design.

1.4. Motivation for Automation

Automation was proposed to address several manufacturing challenges observed in manual production systems.

Manual handling operations often introduce:

- Reduced consistency between production cycles
- Human fatigue during repetitive tasks
- Increased sorting errors
- Limited scalability
- Lower productivity under high demand conditions

The integration of industrial robotics provides several advantages:

- Improved repeatability
- Continuous operation capability
- Faster sorting performance
- Reduced handling variation
- Better quality control
- Easier expansion for future production demands

The selected automation strategy therefore supports both immediate production requirements and long-term manufacturing growth.

2. Proposed Robotic Solution

2.1. Introduction to the Proposed System

To address the operational requirements of Phones-R-Us, an automated robotic sorting and packing workcell was developed using a collaborative UR20 industrial robot integrated with machine vision, vacuum gripping, conveyor handling and safety monitoring systems. The proposed solution was designed to support automated product sorting, defective part rejection and continuous box handling operations while maintaining safe collaborative operation and future manufacturing scalability.

The proposed workcell performs the following primary industrial tasks:

- Automated conveyor-based product handling
- Vision-based colour classification
- Robotic pick-and-place sorting operations
- Defective product rejection
- Automatic full-box transportation
- Safety monitoring and collaborative speed control

The system was designed to satisfy the target throughput requirement of one phone every two seconds while maintaining safe operation, repeatable handling performance and future scalability.

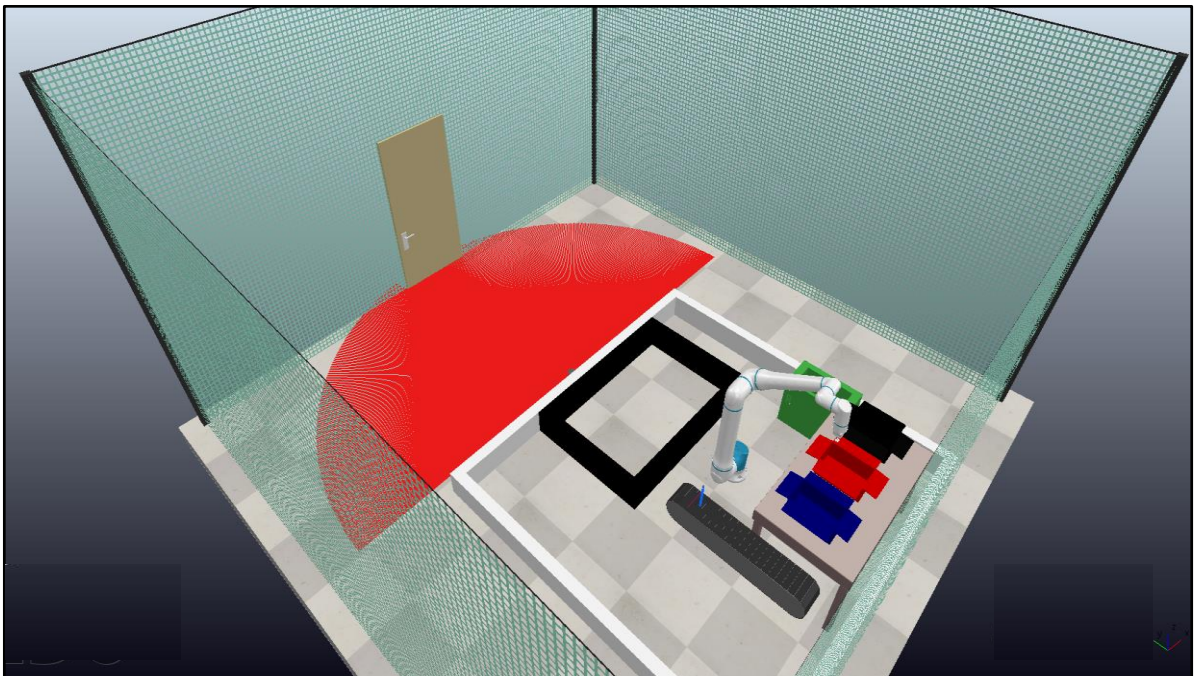


Figure 1: Proposed UR20 robotic sorting and packing workcell implemented within the Coppeliasim environment

2.2. Robot Payload Requirement Analysis

Before selecting the robot manipulator, the maximum handling load was calculated to ensure the proposed system could safely transport fully loaded sorting boxes while maintaining suitable operational safety margins.

Each phone case has a mass of **0.2 kg** and each sorting box stores **63** products. Therefore, the total product mass becomes:

$$m_{products} = 63 \times 0.2 = \mathbf{12.6kg}$$

The selected **VGC10** vacuum gripper possesses a mass of **0.814 kg**. Hence, the total payload requirement becomes:

$$m_{total} = 12.6 + 0.814 = \mathbf{13.414 kg}$$

The calculated payload m_{total} represents the maximum load condition during transportation of a fully filled box from the sorting area to the collection zone.

Although the primary sorting task involves handling individual phone cases, the robot was additionally required to transport fully loaded sorting boxes to the collection area. Consequently, robot selection was based on the maximum payload condition rather than the individual product handling mass.

2.3. Comparative Evaluation of Universal Robots Manipulators

Following payload analysis, multiple Universal Robots platforms were evaluated using payload capability, workspace reach, collaborative suitability and future scalability criteria.

Robot	Payload	Reach	Assessment
UR3e	3 kg	500 mm	payload and reach insufficient
UR5e	5 kg	850 mm	payload and workspace inadequate
UR7e	7.5 kg	850 mm	payload below required handling load
UR10e	12.5 kg	1300 mm	payload lower than required total load
UR12e	12.5 kg	1300 mm	Not payload margin
UR16e	16 kg	900 mm	Payload acceptable but reach inadequate for full cell coverage
UR20	20 kg	1750 mm	satisfies payload, reach and scalability requirements
UR30	30 kg	1300 mm	Technically feasible but oversized and economically unjustified

Table 2: Comparison of Candidate Robots for Phones-R-Us Automation Cell

The analysis confirmed that robots below the **UR16e** could not satisfy the required payload condition of **13.414 kg**. Although the UR16e satisfied the payload requirement, its **900 mm** reach was considered insufficient for reliable access between the conveyor pickup location, sorting boxes, rejection area and collection zone.

The **UR20** provided the most suitable balance between payload capability, workspace coverage, collaborative operation and future scalability while avoiding unnecessary oversizing of the robotic workcell. Detailed manufacturer specifications and supplementary robot comparison data are provided in **Appendix A**.

2.3.1. UR20 Technical Specifications

The UR20 collaborative industrial robot was selected as the primary manipulator for the proposed automation system.



Figure 2: UR20 collaborative industrial robot selected for the proposed automation system

Parameter	Value	Engineering Relevance
Degrees of freedom	6	Supports flexible pick-and-place motion
Payload	20 kg	Exceeds required handling load
Maximum reach	1750 mm	Covers full workcell without repositioning
Robot type	Collaborative robot	Supports safer human interaction
Repeatability	± 0.1 mm	Enables accurate product placement
Joint type	6-axis revolute	Improves workspace accessibility
Maximum TCP speed	Up to 5 m/s	Suitable for high-speed handling
Parameter	Value	Engineering Relevance

Table 3: Key technical specifications of the UR20 collaborative robot

The UR20 manipulator satisfied both the payload and workspace requirements identified during system analysis. The robot provided sufficient reach to access the conveyor pickup location, sorting boxes, recycle area and collection zone without repositioning the robot base. In addition, the six-axis configuration enabled flexible end-effector orientation during pick-and-place handling operations while collaborative functionality supported integration with the proposed safety monitoring system. Complete manufacturer datasheets and supplementary technical specifications are provided in **Appendix A**.

2.4. End Effector Selection

Several gripping technologies were evaluated for the handling of mobile phone cases, including mechanical finger grippers, parallel jaw grippers, and vacuum gripping systems.

Mechanical and jaw-based grippers were not selected because the application required accurate placement of phone cases inside confined cardboard packaging boxes. As the number of stored products increased, the available insertion clearance reduced significantly, limiting the accessibility of finger-based gripping mechanisms and increasing the likelihood of interference with neighbouring products during placement operations.

Vacuum gripping was therefore selected because the end effector approached the product vertically while requiring minimal lateral clearance during insertion. This improved placement repeatability, collision avoidance and accessibility within partially filled boxes. In addition, the lightweight construction and relatively flat surface geometry of the phone cases made them highly suitable for vacuum-based handling operations.

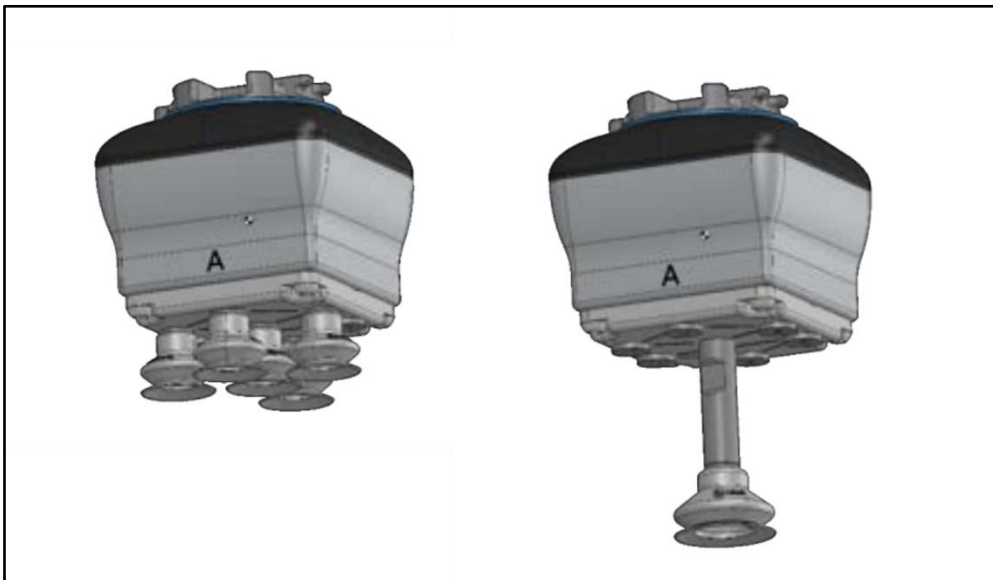


Figure 3: OnRobot VGC10 electrical vacuum gripper selected for the proposed robotic handling system

The selected vacuum gripping approach additionally supported reliable repeated insertion into multi-layer storage boxes containing up to 63 phone cases.

2.4.1. Selected Gripper Configuration

The proposed system employs the **OnRobot VGC10** electrical vacuum gripper integrated with the UR20 collaborative manipulator.

The VGC10 gripper was selected due to its compact design, integrated vacuum generation and suitability for collaborative robotic applications. The elimination of external compressed air systems additionally reduced system complexity and maintenance requirements while supporting easier integration within the proposed workcell.

Although the VGC10 supports multiple suction cups, only a single **30 mm** vacuum cup was implemented within the simulation because the system primarily handled individual phone

cases weighing 200 g each. The single-cup configuration reduced end-effector complexity while maintaining sufficient gripping capability for individual product handling.

The multi-cup capability of the VGC10 remains beneficial for future industrial implementation, particularly during transportation of fully loaded sorting boxes. However, dynamic tool replacement was not implemented within the simulation environment and the same end-effector configuration was therefore retained throughout all operations.

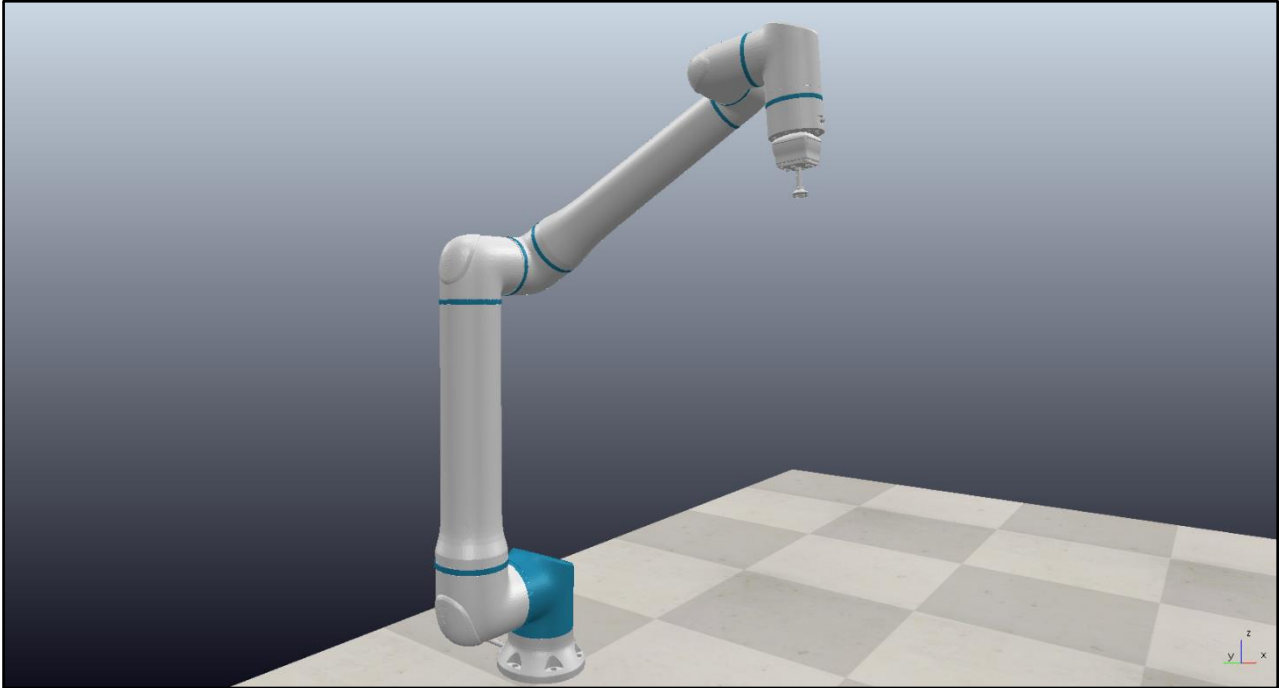


Figure 4: UR20 collaborative robot integrated with the OnRobot VGC10 vacuum gripper

2.4.2. Technical Specifications of Selected Gripper

Parameter	Value
Gripper model	OnRobot VGC10
Gripper type	Electrical vacuum gripper
Maximum payload	15 kg
Gripper mass	814 g
Maximum suction cups	7
Vacuum cup diameter	30 mm
External compressed air	Not required

Table 4: Key technical specifications of the OnRobot VGC10 vacuum gripper

Complete manufacturer datasheets and supplementary gripper specifications are provided in **Appendix B**.

2.4.3. Gripper Payload Verification

The maximum load transported during box handling consists of the filled cardboard box and stored phone cases.

Phone case load: $63 \times 0.2 = 12.6 \text{ kg}$

Box weight: 0.35 kg

Therefore:

$$\text{Total handled object load} = 12.6 + 0.35 = \mathbf{12.95 \text{ kg}}$$

The selected **OnRobot VGC10** vacuum gripper supports a maximum payload of **15 kg**. Therefore:

$$\mathbf{12.95 < 15}$$

The selected gripper therefore satisfied the required handling condition while maintaining an additional payload safety margin for stable operation

2.5. Sensor and Perception System

The proposed robotic workcell employed a perception and safety subsystem to support product identification, conveyor synchronisation and collaborative operation. The sensing architecture integrated a vision sensor for colour classification, a proximity sensor for conveyor control and a SICK S300 safety scanner for operator monitoring.

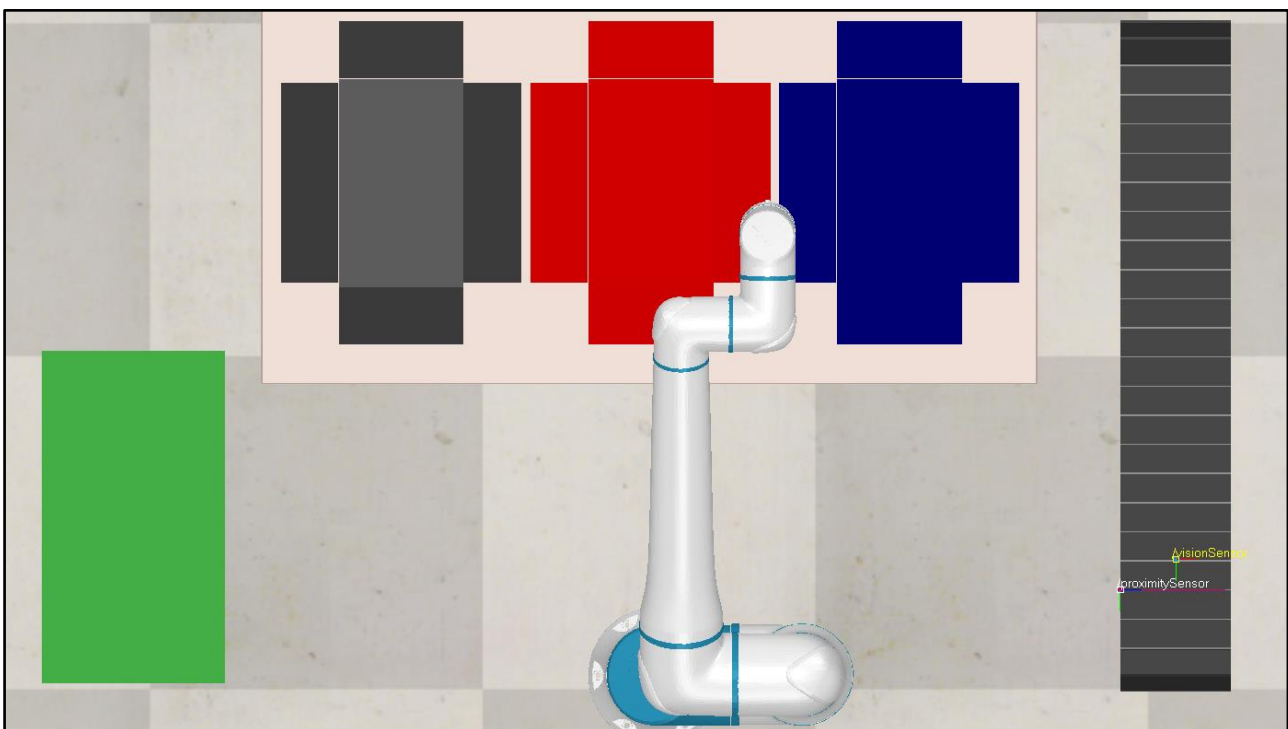


Figure 5: Sensor placement and inspection arrangement within the proposed robotic workcell

The **vision sensor** was positioned above the conveyor inspection region to perform colour classification of incoming phone cases as red, blue or black. Products outside the predefined categories were treated as defective and redirected toward the recycle area. A **proximity sensor**

installed at the inspection point detected incoming products and temporarily stopped the conveyor to provide a stable inspection and pick location for the robotic manipulator.

A **SICK S300** safety scanner was integrated to support collaborative operation within the robotic workcell. Robot speed was adjusted dynamically according to predefined **SAFE**, **WARNING** and **DANGER** monitoring zones to improve operator safety during human interaction.

Sensor	Primary Function
Vision sensor	Colour classification
Proximity sensor	Conveyor detection and synchronisation
SICK S300 scanner	Operator safety monitoring

Table 5: Sensors implemented within the proposed robotic workcell

2.6. Safety System Design

The proposed robotic workcell incorporated an active safety system to support collaborative human–robot interaction during production and maintenance operations. Safety monitoring was achieved using a **SICK S300** safety scanner integrated with robot speed control, collaborative operating modes and emergency stop logic.

2.6.1. Safety Zone Configuration

Safety Zone	Distance Range	Robot Behaviour	Speed Factor
Safe	> 2 m	Normal operation	1.0
Warning	0.5–2 m	Reduced speed	0.2
Danger	< 0.5 m	Emergency stop	0

Table 6: Distance-based safety response strategy implemented within the robotic workcell

The implemented safety logic continuously monitored operator distance and dynamically adjusted robot motion according to predefined monitoring zones. Speed reduction was applied when personnel entered the warning region, while robot motion was stopped completely within the danger zone.

2.6.2. Collaborative Operating Mode

The system supports two operating modes:

Mode	Description	Speed Behaviour
Normal mode	Standard production operation	User-adjustable speed
Collaborative mode	Human interaction operation	Fixed reduced speed

Table 7: Operating modes implemented within the robotic workcell

The implemented safety architecture combined distance-based monitoring, collaborative speed reduction and emergency stop functionality to support safer operation within the proposed robotic workcell.

2.7. Motion Planning and Workspace Design

2.7.1 Workcell Layout and Robot Workspace

The proposed robotic workcell was designed to support automated handling, sorting, defective product rejection and final box transportation operations within a single collaborative workspace. The UR20 manipulator was positioned centrally to maximise workspace utilisation and maintain direct access between the conveyor pickup location, sorting boxes, recycle area and collection zone without repositioning the robot base.

2.7.2. Approach Point Strategy

Intermediate approach points were implemented above operational task locations to create controlled vertical entry motions before pickup and placement operations. This strategy improved collision avoidance, positioning repeatability and motion consistency during sorting and box handling tasks.

Approach Point	Purpose
approachPickingPoint	Phone pickup approach
approachRedBox	Red phone case placement and box pickup
approachBlueBox	Blue phone case placement and box pickup
approachBlackBox	Black phone case placement and box pickup
approachRecycleBin	Defective part disposal
approachCollectionArea	Full box placement

Table 8: Approach points

2.7.3. Robot Path Planning

Robot motion was divided into predefined trajectories to simplify controller implementation and improve operational repeatability. Separate paths were implemented for sorting operations, defective product handling and full-box transportation between the sorting area and collection zone. The robot selected trajectories according to colour classification results generated by the vision subsystem. Dedicated trajectories between the pickup location and individual colour sorting boxes were not separately defined because these motions already represented the most direct and efficient paths within the workcell layout.

Path	Purpose
Path 1	Home → Picking point
Path 2	Picking point → Recycle bin
Path 3	Recycle bin → Picking point
Path 4	Blue Box → Collection Area
Path 5	Red Box → Collection Area
Path 6	Black Box → Collection Area

Table 9: Predefined robot motion trajectories implemented within the proposed workcell

The predefined trajectory strategy simplified controller implementation while improving operational repeatability during sorting and box transportation tasks.

2.8. Control Architecture and System Logic

2.8.1. Overall Operational Sequence

The proposed robotic system operated using event-driven control logic beginning from phone case detection on the conveyor and ending at placement of filled boxes within the collection area. The operational sequence included conveyor stopping, colour inspection, robotic sorting, defective product rejection and automatic full-box transportation. The overall operational workflow implemented within the robotic system is illustrated in *Figure 6*.

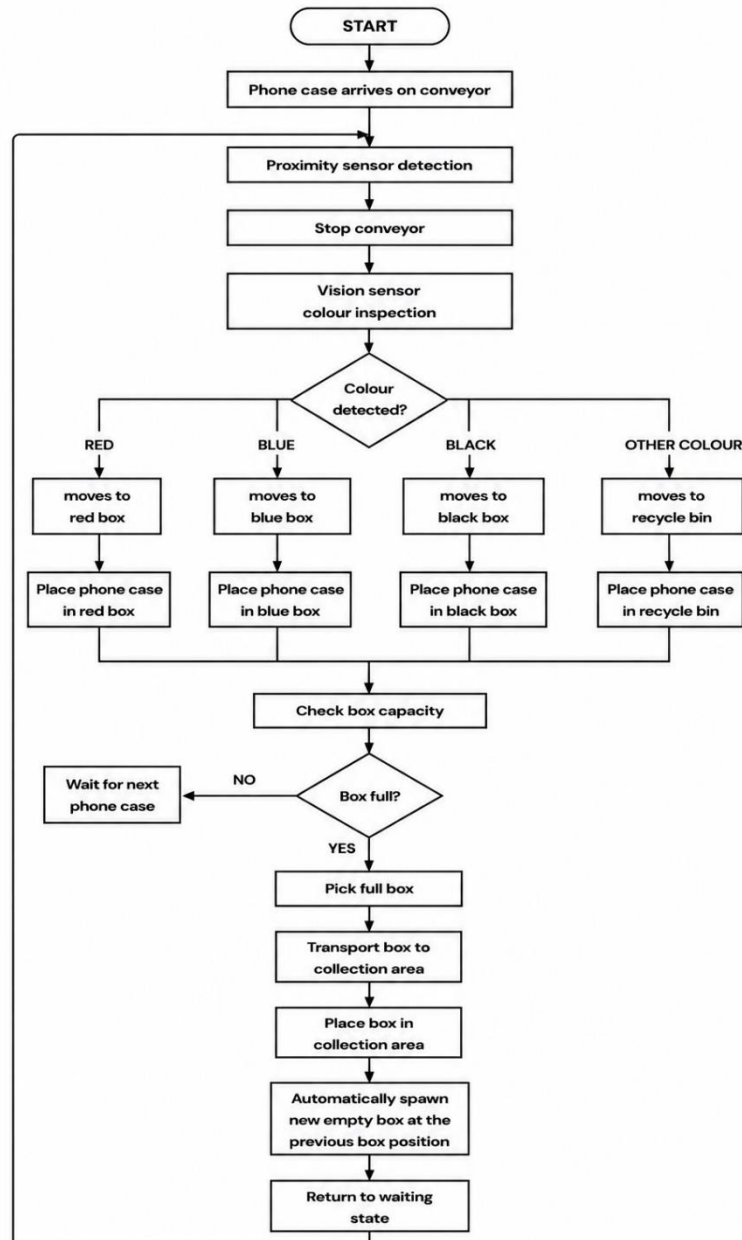


Figure 6: Overall operational sequence implemented within the robotic sorting system

The implemented sequence enabled continuous sorting operations while maintaining repeatable robot motion and automated box handling throughout the production cycle.

Supporting implementation scripts for robot control, user interface logic, proximity sensing and vision processing are provided in **Appendices B–E**.

2.8.2. Finite State Machine (FSM) Control

A finite state machine (FSM) architecture was implemented to coordinate robot motion, sorting operations, defective product handling and automatic box transportation. The FSM structure improved motion organisation, operational repeatability and system scalability while simplifying state-based controller implementation. The implemented finite state machine structure used for robot coordination and box handling operations is shown in Figure 7.

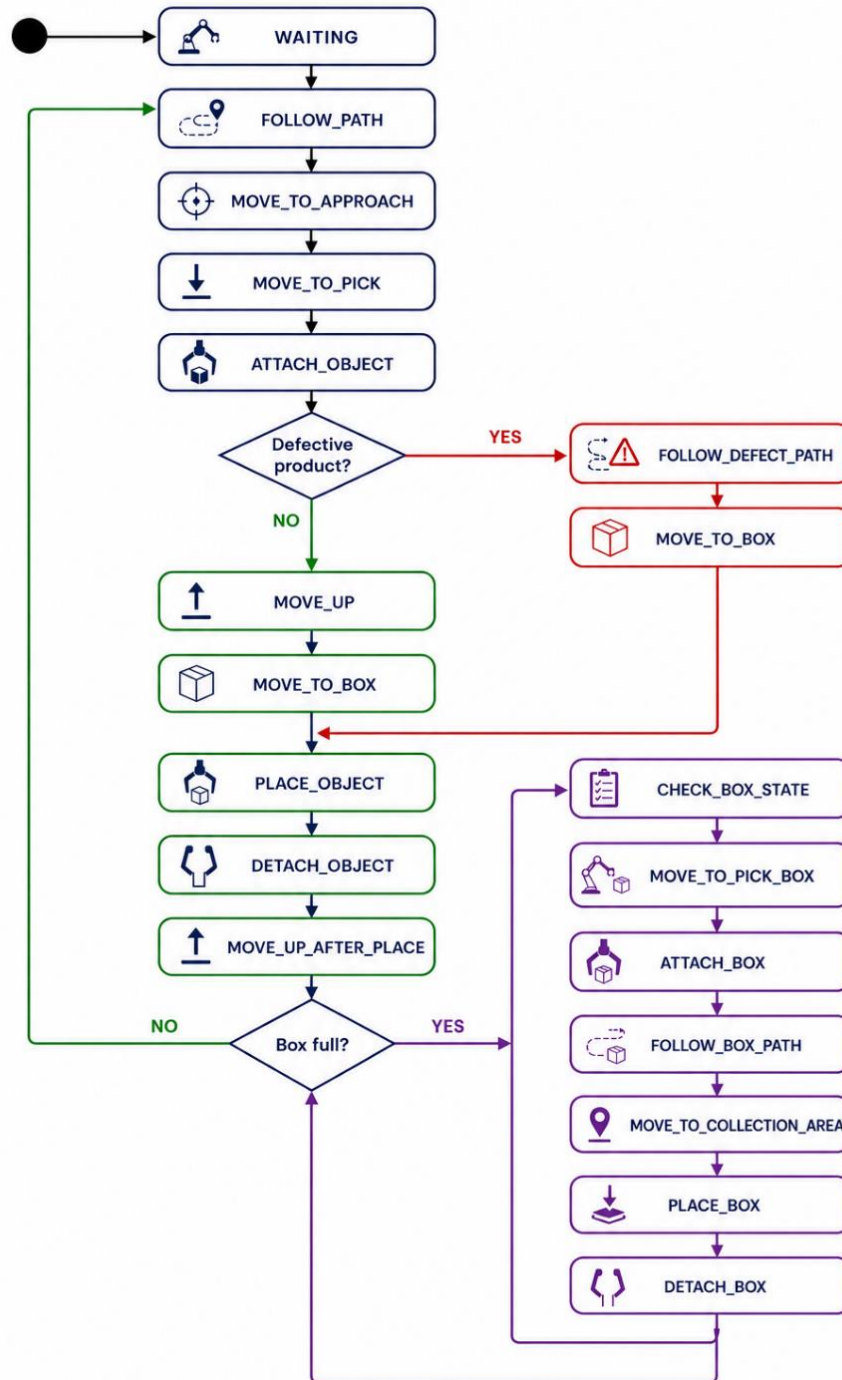


Figure 7: Finite state machine for sorting and box handling operation

2.8.3. Control Architecture

The control architecture integrated proximity sensing, vision-based colour classification, FSM coordination, robot motion planning and safety supervision within a unified control framework. Automatic box replacement logic enabled continuous production by transporting full boxes to the collection area and introducing new empty boxes when required. The integrated sensing, control and robot coordination architecture is illustrated in Figure 8.

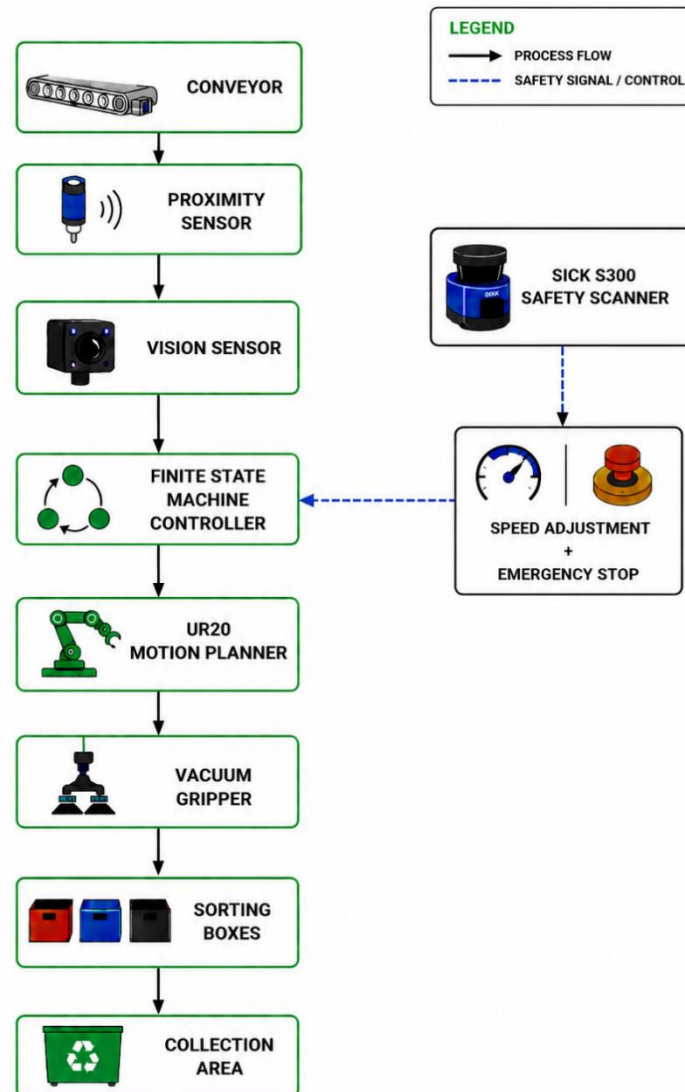


Figure 8: Integrated control architecture of the proposed robotic sorting and packing system

Additional flowcharts for colour classification and automatic box handling logic are provided in Appendix F.

3. Implementation and Simulation Results

3.1. Simulation Environment Setup

The proposed robotic sorting and packing system is implemented in CoppeliaSim to evaluate robot motion, sensing operations, sorting behaviour and automatic box handling within a virtual production environment. The developed CoppeliaSim environment used for implementation and testing is shown in Figure 9.

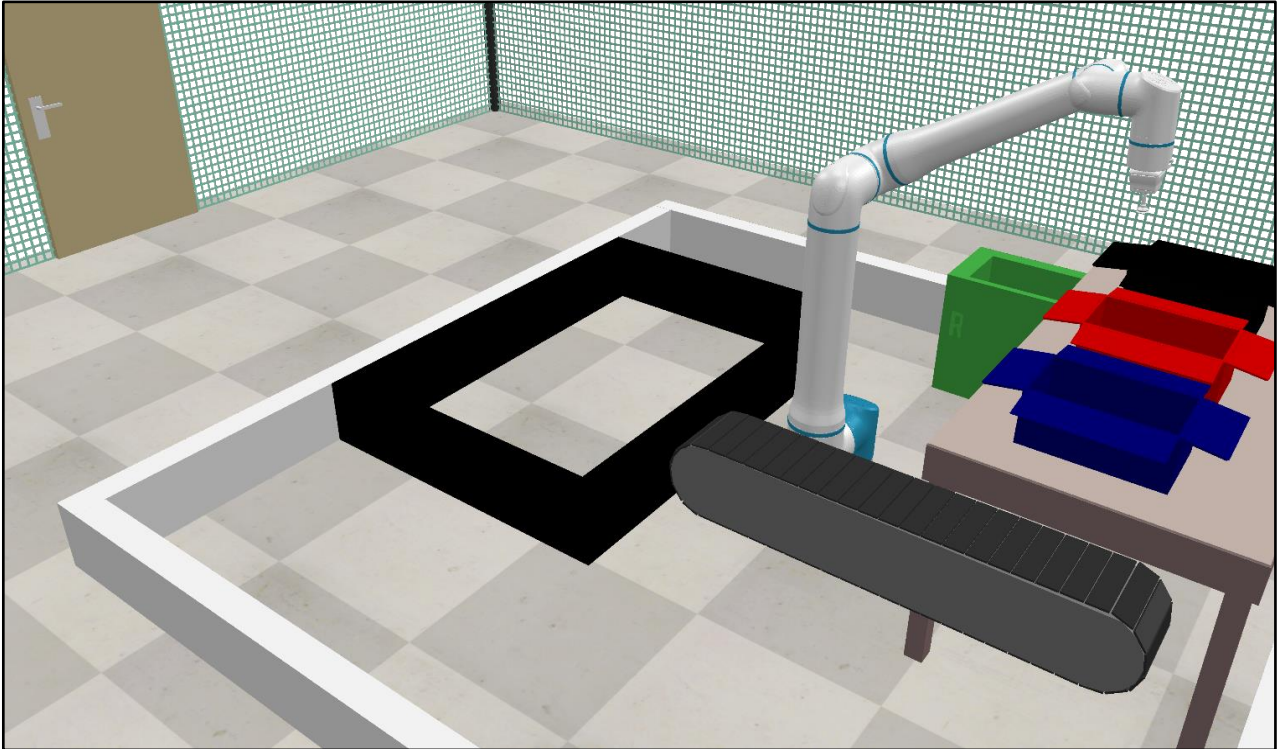


Figure 9: Complete CoppeliaSim implementation of the Phones-R-Us robotic workcell

The simulation environment is developed using the previously defined system architecture and included the conveyor system, robot workspace, sorting boxes, recycle area, collection region, sensing devices and safety elements.

3.2. System Implementation

The proposed automation system is implemented using multiple scripts to coordinate robot motion, user interaction, sensing operations and colour classification within the simulation environment. The implementation is divided into separate modules to simplify control development and improve system organisation.

Script	Function	Appendix Reference
Robot Main Script	Controls robot motion, sorting logic, box handling and FSM execution	Appendix B
Main UI Script	Handles user interaction and process control	Appendix C
Proximity Sensor Script	Detects incoming phone cases and controls conveyor stopping	Appendix D
Vision Sensor Script	Performs colour detection and classification	Appendix E

Table 10: Control scripts implemented within the CoppeliaSim environment

The robot main script coordinated robot motion, sorting decisions, defective product handling and box transportation operations. Additional scripts were implemented for user interaction, proximity-based conveyor control and vision-based colour classification. The modular implementation strategy simplified debugging, improved code organisation and enabled independent development of sensing, motion and control functions. Detailed script implementations are provided in **Appendices B–E**.

3.3. Sorting Operation Results

The implemented robotic system was tested using repeated sorting cycles within the CoppeliaSim environment to validate sensing accuracy, robot motion coordination and placement reliability. The testing sequence included conveyor detection, colour classification, robotic pickup and placement into corresponding sorting locations.

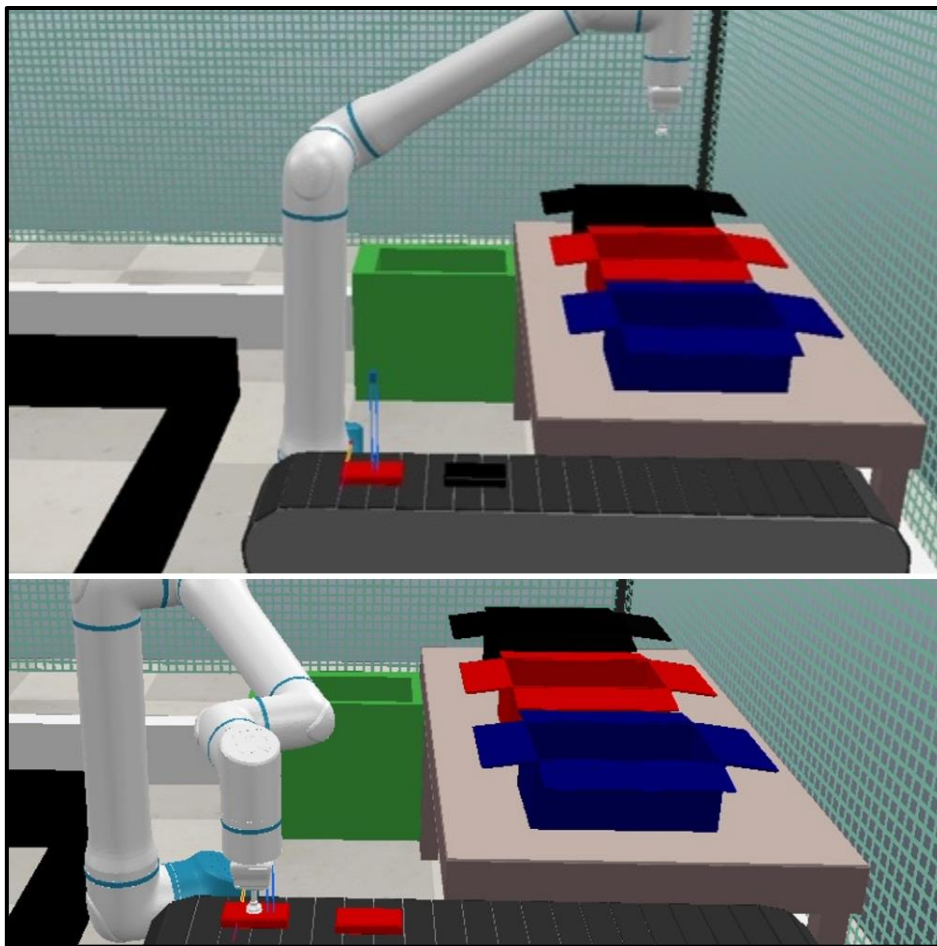


Figure 10: Conveyor inspection and robotic pickup sequence within the sorting process

Figure 10 illustrates the inspection and robotic pickup stages of the implemented sorting sequence. The proximity sensor detected incoming phone cases at the inspection position and temporarily stopped the conveyor to support stable image acquisition and repeatable robot pickup operations. Following colour classification, the UR20 robot moved to the predefined pickup location and attached the phone case using the vacuum gripper.

3.3.1. Color-Based Sorting Validation

The implemented sorting algorithm successfully classified and sorted red, blue and black phone cases into their corresponding storage boxes. Products outside the predefined colour thresholds were automatically redirected toward the recycle area.

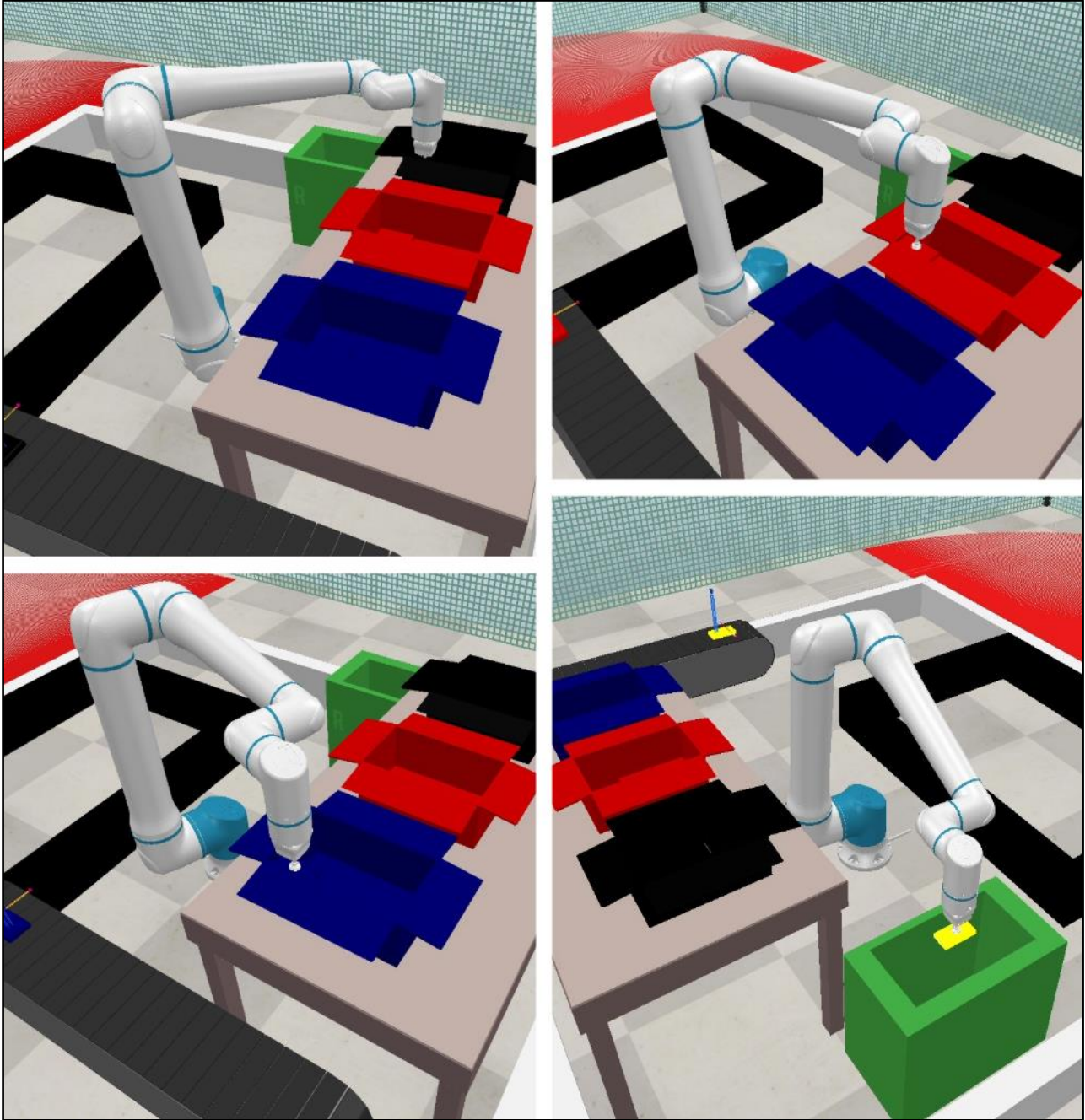


Figure 11: Validation of colour-based sorting and defective product rejection operations

The results demonstrated effective coordination between the vision sensor, finite state machine controller and robot motion planner during repeated sorting operations.

3.3.2 Box Filling and Placement Arrangement

Phone cases were arranged systematically within each sorting box to maximise storage capacity and support stable stacking behaviour. Once the predefined box capacity was reached, the UR20 robot automatically transported the filled box to the collection area and introduced a new empty box to maintain continuous production operation.

Additional figures showing box filling arrangements and automatic box transportation sequences are provided in **Appendix G**.

3.4. User Interface and System Monitoring

A dedicated graphical user interface (GUI) was developed within CoppeliaSim to support operation, testing and monitoring of the robotic sorting system. The interface enabled the operator to control production execution, modify operating parameters and monitor live sorting performance during simulation.

The implemented GUI included controls for system start/stop operation, automatic production mode selection, collaborative operating mode and robot speed adjustment. In automatic mode, phone cases were continuously generated and processed without manual intervention, while manual spawning controls were additionally provided for debugging and validation of the colour classification system.

Two operating modes were implemented within the interface. Normal mode operated the robot at full production speed, while collaborative mode reduced robot speed to support safer human – robot interaction conditions. The applied robot speed could also be adjusted directly through the interface controls.

The GUI additionally displayed live production counters for red, blue, black and defective phone cases together with completed box counts transferred to the collection area. Real-time safety status monitoring was integrated with the SICK S300 safety scanner to provide continuous indication of workcell operating conditions during simulation execution.

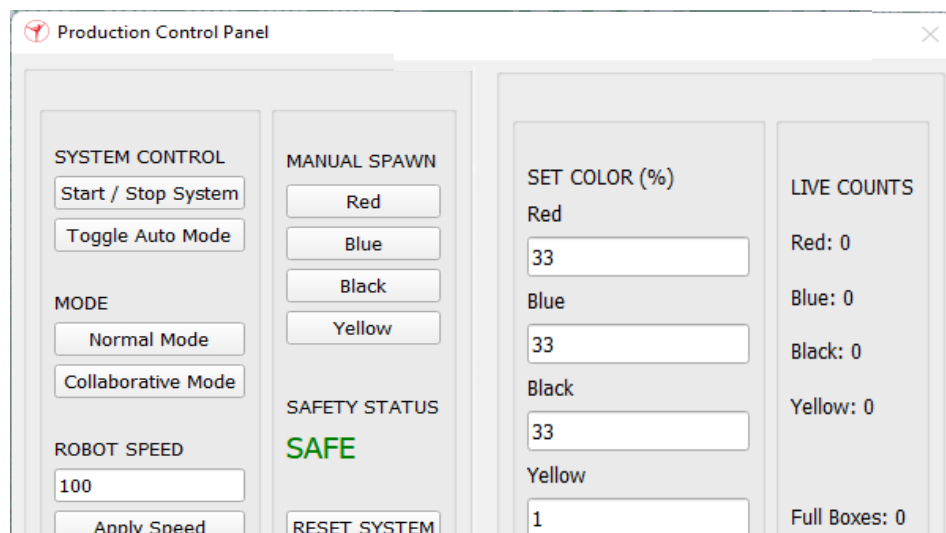


Figure 13: Graphical user interface developed for robotic system operation and monitoring

4. MATLAB Kinematic Analysis

4.1. MATLAB Environment and Robot Model

MATLAB was used to perform forward kinematic (FK), inverse kinematic (IK), workspace and motion validation for the proposed UR20 robotic sorting system. The UR20 industrial robot was imported into MATLAB using a URDF model generated through ROS-based modelling tools, enabling accurate representation of the robot geometry, joint structure and motion behaviour within the analysis environment.

A simplified digital representation of the robotic workcell was recreated within MATLAB using collision objects representing the conveyor system, sorting boxes, recycle bin, collection area, workstation table and surrounding workspace elements. The reconstructed environment closely matched the layout implemented in CoppeliaSim and was used to validate task reachability, robot accessibility and motion feasibility before simulation testing.

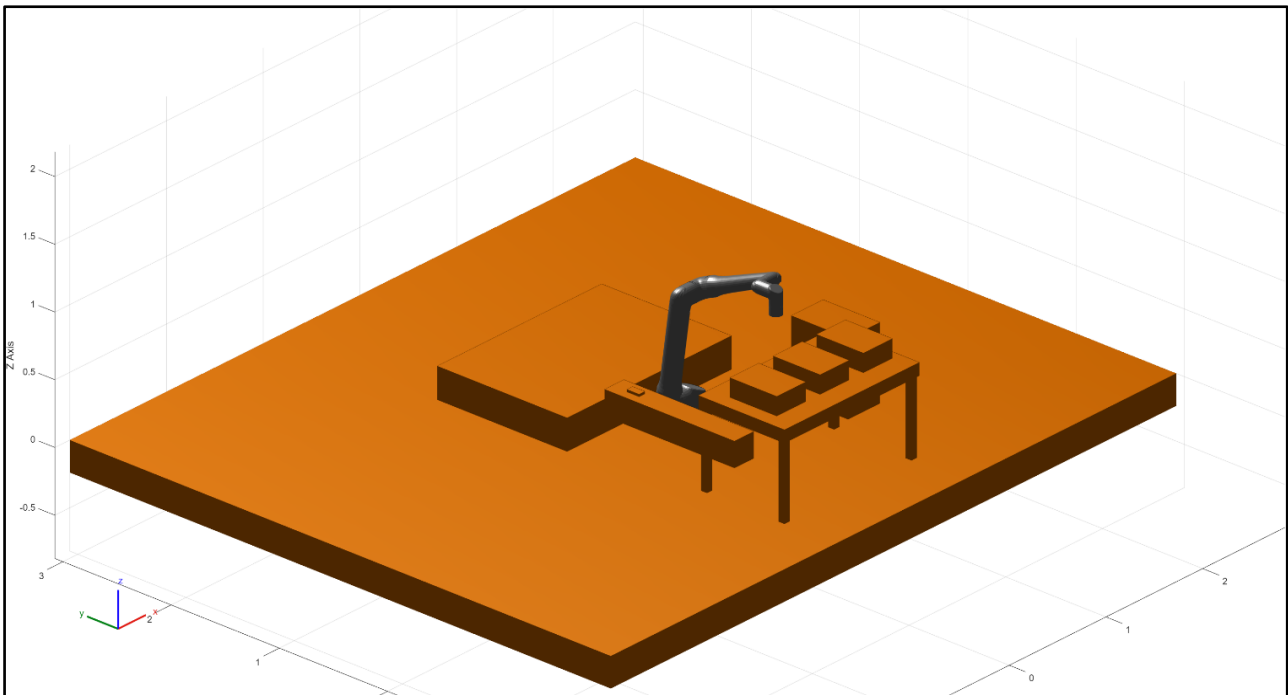


Figure 14: Reconstructed UR20 robotic workcell environment in MATLAB using imported URDF and collision objects

MATLAB IK Designer was used to generate and validate industrial task poses required for the sorting operation. Validated poses included conveyor pickup positions, sorting box placements, recycle handling configurations and collection area transport poses. A total of sixteen task poses were generated and later integrated into the motion planning strategy implemented within CoppeliaSim.

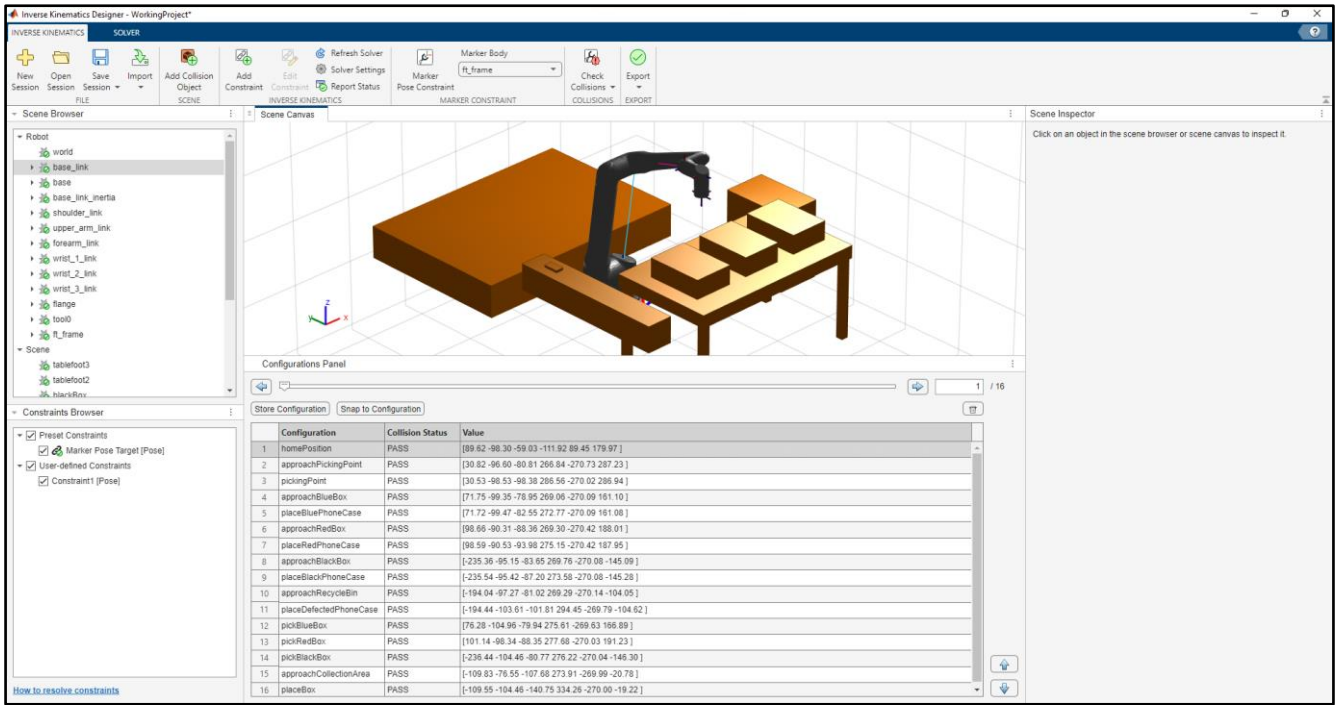


Figure 15: MATLAB IK Designer environment showing validated UR20 industrial task poses

The MATLAB environment provided an additional validation stage before implementation within CoppeliaSim. The generated kinematic results influenced robot base positioning, waypoint selection and overall workspace layout to ensure collision-free operation and efficient industrial material handling trajectories.

Additional MATLAB setup parameters, collision object definitions and complete pose configurations are provided in **Appendix F**.

4.2 Forward Kinematics Analysis

Forward kinematic analysis was performed in MATLAB to determine the Cartesian position of the UR20 robot tool centre point (TCP) at validated industrial task configurations. The analysis was used to verify that the robot could accurately reach the required pickup, sorting, recycle and collection positions within the proposed workcell layout.

The imported UR20 robot model was used to compute TCP positions for all predefined task poses generated within MATLAB IK Designer. Homogeneous transformation matrices were evaluated using the robot joint configurations to obtain the TCP coordinates relative to the robot base frame.

$$T_n^0 = T_1^0 T_2^1 T_3^2 \dots T_n^{n-1}$$

The calculated positions were then analysed to verify spatial accessibility and task feasibility throughout the robotic workcell.

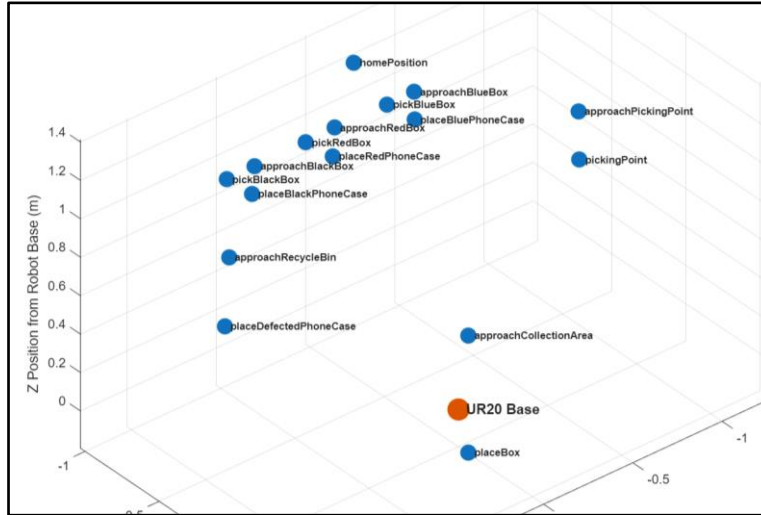


Figure 16: Forward kinematic TCP positions generated for validated industrial task poses

Figure 16 illustrates the calculated TCP positions for all major industrial task locations including conveyor pickup, colour sorting, recycle handling and collection area operations. The generated positions confirmed that the selected robot base placement enabled efficient access to all operational regions while maintaining suitable separation between task locations.

The forward kinematic analysis directly influenced the final workstation layout implemented within CoppeliaSim. The calculated TCP positions were used to optimise robot base positioning, reduce unnecessary robot travel and support the generation of smooth industrial motion trajectories between task locations.

Complete TCP coordinate outputs and additional forward kinematic calculations are provided in **Appendix G**.

4.3 Inverse Kinematics Analysis

Inverse kinematic analysis was performed in MATLAB to validate the feasibility of the major industrial task poses required within the robotic sorting system. The analysis was used to determine the joint-space configurations necessary for conveyor pickup, colour sorting, recycle handling and full-box transportation operations.

The MATLAB inverse kinematics solver was applied to the imported UR20 robot model using the validated Cartesian task poses generated during workspace planning. Each target pose was solved relative to the robot base frame to obtain feasible joint configurations while maintaining realistic industrial robot orientations for material handling operations.

Pose	IK_Status	FK_IK_Error
"homePosition"	"success"	0
"approachPickingPoint"	"success"	4.2753e-08
"pickingPoint"	"success"	3.4123e-08
"approachBlueBox"	"success"	2.1499e-08
"placeBluePhoneCase"	"success"	3.9328e-08
"approachRedBox"	"success"	1.7838e-08
"placeRedPhoneCase"	"success"	8.9051e-09
"approachBlackBox"	"success"	5.5355e-08
"placeBlackPhoneCase"	"success"	3.5672e-09
"approachRecycleBin"	"success"	3.5685e-08
"placeDefectedPhoneCase"	"success"	7.0451e-09
"pickBlueBox"	"success"	3.0175e-08
"pickRedBox"	"success"	2.7133e-08
"pickBlackBox"	"success"	1.4277e-08
"approachCollectionArea"	"success"	1.8005e-08
"placeBox"	"success"	2.2952e-08

Figure 17: Inverse kinematic validation results for industrial task poses

The inverse kinematic results confirmed successful convergence for all major task poses used within the robotic workcell. Very small FK–IK error values were obtained for all configurations, validating the accuracy of the generated joint solutions and confirming the feasibility of the proposed industrial handling operations.

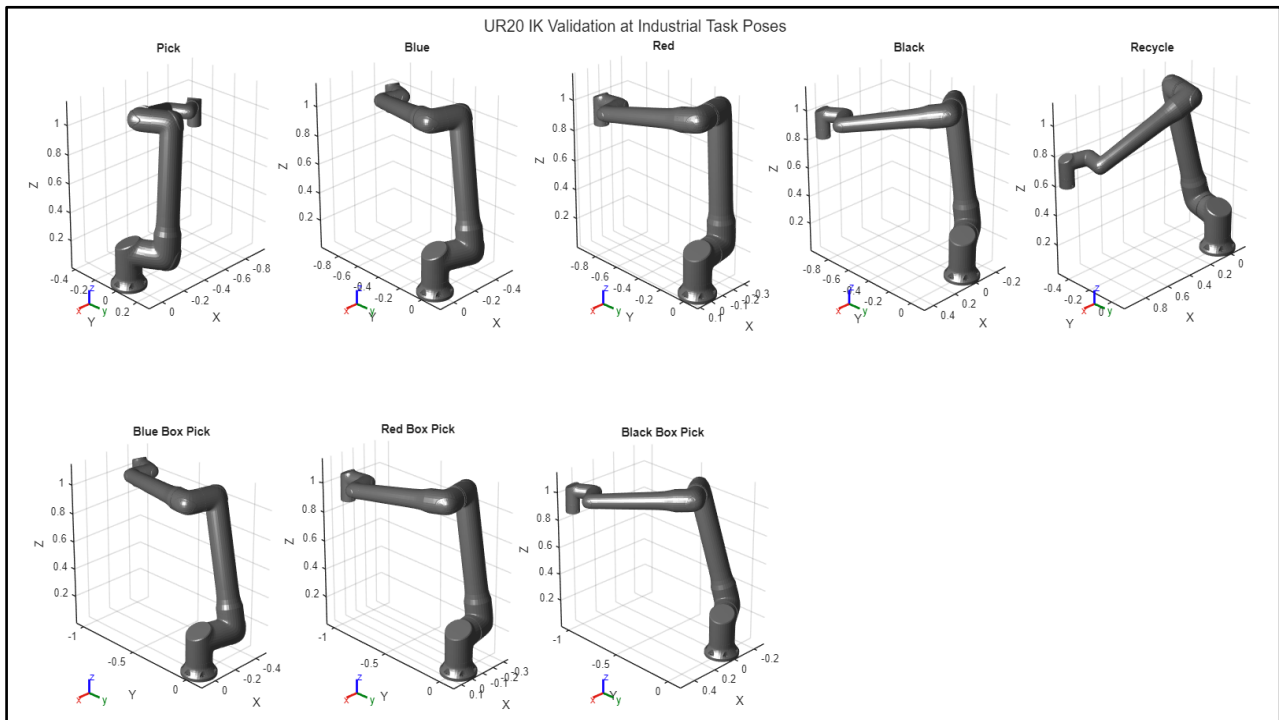


Figure 18: Validated UR20 inverse kinematic configurations for industrial sorting and handling operations

Figure 18 demonstrates that all required task poses were reachable while maintaining practical robot postures and collision-free accessibility within the workcell environment. Intermediate approach poses were additionally implemented to improve trajectory smoothness and reduce the risk of collisions during industrial handling operations.

The generated inverse kinematic solutions were later integrated into the CoppeliaSim motion planning framework to support trajectory generation between pickup, placement, recycle and

collection operations. The validated joint configurations also assisted in defining safe approach directions and optimised robot motion paths throughout the sorting cycle.

Complete joint-space configurations and additional inverse kinematic outputs are provided in **Appendix H**.

4.4 Workspace and Reachability Validation

Workspace and reachability analysis was performed to validate that all required industrial task locations were accessible within the UR20 robot operating region. The analysis was used to verify conveyor pickup feasibility, sorting box accessibility, recycle handling operations and transportation of filled boxes to the collection area.

A workspace verification study was conducted in MATLAB using randomly generated robot configurations to approximate the reachable operating region of the UR20 robot. The generated workspace cloud was then compared against the predefined industrial task points used within the robotic sorting system.



Figure 19: Three-dimensional UR20 workspace validation showing reachable operating volume and validated industrial task targets

The generated workspace cloud confirmed that all industrial task locations were positioned safely within the reachable operating region of the UR20 robot. Sufficient reach margins were maintained between the robot maximum reach boundary and all operational task points, reducing the risk of singular configurations or excessive joint extension during material handling operations.

TaskPoint	DistanceFromBase_m	ReachMargin_m	Reachable
"Pick phone case"	1.2297	0.52027	"Yes"
"Approach pick"	1.3931	0.35689	"Yes"
"Place blue case"	1.3238	0.42617	"Yes"
"Place red case"	1.2214	0.52861	"Yes"
"Place black case"	1.2783	0.47172	"Yes"
"Place defect"	1.1906	0.55944	"Yes"
"Pick blue box"	1.3917	0.35828	"Yes"
"Pick red box"	1.3254	0.42456	"Yes"
"Pick black box"	1.4064	0.34355	"Yes"
"Approach collection"	1.3454	0.40464	"Yes"
"Place full box"	1.0078	0.74222	"Yes"

Figure 21: Reachability validation analysis table

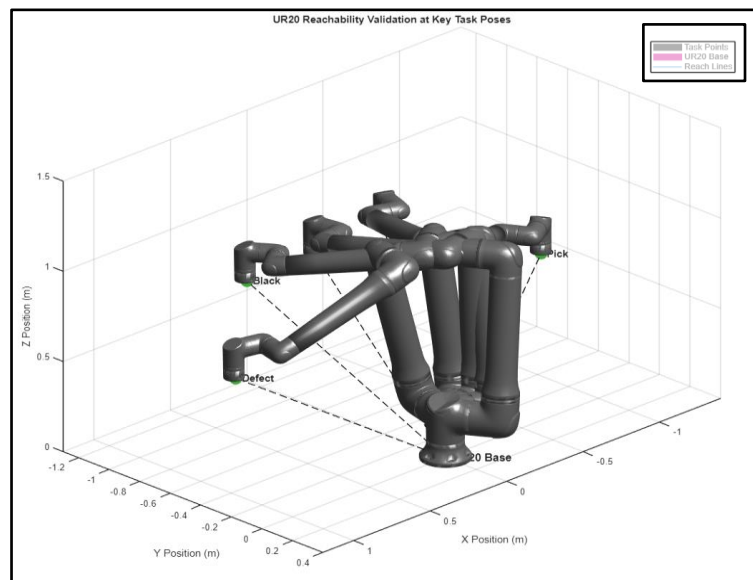


Figure 22: Reachability validation of major industrial handling poses within the UR20 workcell

The reachability analysis confirmed that the selected robot base position provided effective access to all operational areas while maintaining suitable workspace separation between conveyor, sorting and collection tasks. The results also demonstrated that the implemented approach poses improved accessibility and supported smoother industrial motion planning between task locations.

The workspace analysis directly influenced the final arrangement of the conveyor system, sorting boxes, recycle bin and collection area within the CoppeliaSim environment. The validated layout reduced unnecessary robot travel while ensuring reliable reachability for all industrial handling operations.

A supplementary top-view workspace representation used for additional workspace validation is provided in **Appendix I**.

4.5 Cartesian Motion Planning and Path Validation

Cartesian motion planning analysis was performed to validate the industrial robot trajectories implemented within the CoppeliaSim sorting environment. The analysis was used to evaluate robot movement between conveyor pickup, colour sorting, recycle handling and collection area operations while maintaining smooth and collision-free industrial material flow.

The Cartesian trajectories shown in Figure 23 represent the actual motion paths executed by the UR20 robot within the implemented CoppeliaSim workcell. The paths were generated using the validated MATLAB task poses and predefined industrial waypoints developed during motion planning. Separate trajectories were implemented for conveyor pickup operations, sorting box placement, recycle handling and transportation of filled boxes to the collection area.

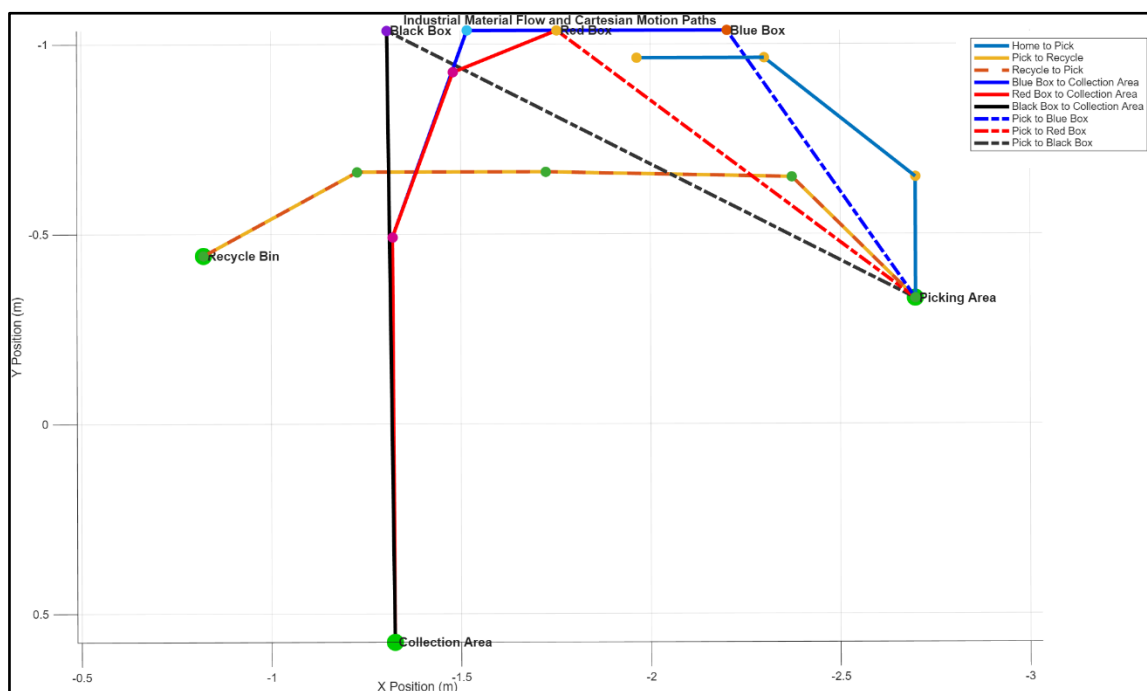


Figure 23: Actual UR20 industrial motion paths implemented within the CoppeliaSim robotic sorting environment

Figure 23 demonstrates the complete industrial material flow implemented within the robotic sorting system. The validated robot trajectories confirmed smooth and collision-free motion between conveyor pickup, sorting, recycle and collection operations. Intermediate approach paths additionally improved trajectory consistency and reduced unnecessary robot movement during continuous sorting cycles.

The validated Cartesian trajectories were integrated directly into the CoppeliaSim robot control framework and formed the basis of the implemented industrial motion planning strategy. The resulting path structure supported reliable material handling, improved robot motion repeatability and efficient workstation operation within the constrained 3×3 m workspace.

5. Safety Assessment and Risk Mitigation

The proposed robotic sorting workcell was designed in accordance with general industrial and collaborative robotics safety principles to minimise operational hazards while maintaining efficient production performance. The workcell integrates a UR20 collaborative manipulator, conveyor transport system, vacuum gripper, machine vision inspection, automatic box handling and a SICK S300 safety scanner within a restricted 3 m × 3 m operating area. As the robot operates in close proximity to conveyors, sorting boxes and potential operator access regions, a structured safety assessment was required to reduce operational risk and support safe human – robot interaction.

The primary hazards identified within the proposed workcell included robot collision risks, pinch points around moving joints and conveyors, dropped objects during transportation, unintended robot movement and crushing hazards during full-box handling operations. These risks are consistent with common industrial robotic hazards identified within ISO 10218-1 and ISO 10218-2 industrial robot safety standards for robot systems and integration. In addition, collaborative operation principles referenced within ISO/TS 15066 were considered during implementation of safety monitoring and speed reduction behaviour.

Hazard	Associated Risk	Implemented Mitigation
Robot collision with operator	Impact or crushing injury	SICK S300 safety scanner with SAFE, WARNING and DANGER zones
Collision with conveyor or fixtures	Damage to robot, conveyor or sorting boxes	Predefined trajectories, approach points and MATLAB workspace validation
Conveyor motion during pickup	Unstable gripping or positioning error	Proximity sensor stops conveyor before inspection and pickup
Dropped phone case or box	Product damage or unstable handling	Vacuum gripper with controlled motion and reduced collaborative speed mode
Transportation of full boxes	Increased payload and crushing hazard	UR20 payload safety margin and predefined collection paths
Unexpected restart or control fault	Unsafe robot movement	FSM-based state control and emergency stop logic

Table 11: Identified hazards and implemented mitigation measures within the robotic workcell

A SICK S300 safety scanner was integrated within the simulation environment to provide active operator monitoring and adaptive robot speed control. The scanner generated predefined SAFE, WARNING and DANGER operating zones around the robot workspace. During normal operation, the robot operated at full programmed speed. When personnel entered the warning zone, robot speed was automatically reduced to 20% of normal operating speed to minimise collision risk. Entry into the danger zone triggered an emergency stop condition which immediately halted robot motion and conveyor operation. This behaviour reflects Speed and Separation Monitoring (SSM) principles commonly used in collaborative robotic systems and discussed within ISO/TS 15066 collaborative robotics guidance.

Additional risk reduction measures were incorporated into the workcell design to improve operational safety and motion predictability. Intermediate approach points were positioned above all operational task locations to ensure controlled vertical entry and exit motions during pickup and placement operations. This strategy reduced collision risk with sorting boxes, conveyors and surrounding fixtures while improving repeatability during continuous operation. Conveyor motion was automatically stopped during colour inspection and pickup operations to maintain stable object positioning before robotic interaction.

The system additionally incorporated emergency stop functionality within the graphical user interface, enabling immediate shutdown of robot and conveyor motion during abnormal operating conditions. Collaborative mode operation reduced robot speed to approximately 0.3 m/s during testing and maintenance tasks to improve operator safety during human interaction. The implemented safety architecture therefore combined sensor-based monitoring, zoning, emergency stopping, collaborative speed reduction and controlled motion planning to support safer and more reliable operation.

The proposed workcell aligns with the general safety expectations outlined within ISO 10218-1 and ISO 10218-2 for industrial robot safety and robot system integration, while collaborative operating behaviour was informed by ISO/TS 15066 guidance for collaborative robotic applications. Emergency stop behaviour and monitored stopping principles were additionally influenced by IEC 60204-1 industrial machinery safety recommendations.

6. PESTLE Analysis

The proposed robotic automation system introduces several political, economic, social, technological, legal and environmental implications for Phones-R-Us. The analysis demonstrates how adoption of robotic automation may influence future manufacturing operations, workforce requirements and long-term business performance.

Factor	General Considerations	Specific Considerations for Phones-R-Us
Political	The UK government continues to promote robotics, automation and Industry 4.0 technologies to improve manufacturing productivity, resilience and digital transformation. National automation strategies encourage increased adoption of collaborative robotics within industrial production environments (Department for Science, Innovation and Technology, 2023).	The proposed UR20 robotic workcell supports automation within UK manufacturing by improving production scalability, reducing reliance on repetitive manual labour and supporting future production expansion for Phones-R-Us. The project also aligns with increasing adoption of collaborative robotic systems within advanced manufacturing sectors.

Economic	Robotics and automation technologies can improve manufacturing productivity, reduce long-term labour costs and increase operational consistency. According to Make UK and techUK, automation technologies are expected to contribute significantly to the UK economy over the next decade (Make UK, 2023; techUK, 2023).	The proposed system reduces dependency on repetitive manual sorting operations while improving throughput consistency and product handling repeatability. Although the UR20 robot, safety scanner and sensing systems introduce additional capital costs, automation may reduce long-term operational costs, handling errors and production bottlenecks during high-volume manufacturing.
Social	Automation can improve workplace safety by reducing repetitive and physically demanding tasks. However, increased automation may also reduce demand for low-skilled labour while increasing demand for technical and supervisory roles (ONS, 2023).	The robotic sorting system reduces repetitive manual handling operations previously performed by human operators. The collaborative workcell allows workers to transition toward monitoring, maintenance and quality supervision roles. Staff retraining and technical upskilling would therefore become increasingly important following deployment of the proposed system.
Technological	Modern manufacturing increasingly relies on robotics, machine vision, intelligent sensing and digital simulation tools to improve flexibility and efficiency. Collaborative robots and virtual commissioning technologies are becoming increasingly important within Industry 4.0 manufacturing systems (Schwab, 2016).	The proposed system integrates collaborative robotics, machine vision, finite state machine control, MATLAB kinematic analysis and CoppeliaSim virtual commissioning within a single automated workcell. The modular design additionally supports future integration of AI-based inspection systems, predictive maintenance, autonomous mobile robots (AMRs) and advanced safety monitoring technologies.

Legal	Industrial robotic systems must comply with recognised international safety standards governing robot operation, safeguarding and collaborative behaviour. ISO 10218-1 and ISO 10218-2 define safety requirements for industrial robots and robot system integration, while ISO/TS 15066 provides guidance for collaborative robotic operation including speed and separation monitoring, force limitation and workspace safety requirements. IEC 60204-1 additionally defines electrical safety and emergency stop principles for industrial machinery.	The proposed workcell was designed with consideration of ISO 10218-1, ISO 10218-2 and ISO/TS 15066 collaborative robotics guidance. The implemented SICK S300 scanner, emergency stop logic, collaborative speed reduction and predefined SAFE, WARNING and DANGER zones support alignment with accepted industrial safety practices for collaborative robotic operation.
Environmental	Robotics and automation can support sustainable manufacturing by improving consistency, reducing product waste and minimising unnecessary energy consumption. Virtual commissioning also reduces physical prototyping requirements during development.	The proposed system reduced unnecessary conveyor operation by stopping transport during inspection and pickup operations. Virtual commissioning within CoppeliaSim reduced the need for repeated physical testing and material waste during development. Future optimisation of robot trajectories and energy-efficient motion planning could further improve operational sustainability.

The PESTLE analysis demonstrates that the proposed robotic system provides significant advantages in productivity, operational consistency, scalability and workplace safety while also introducing important workforce, legal and economic considerations. The project reflects modern Industry 4.0 manufacturing trends where collaborative robotics, intelligent sensing and virtual commissioning technologies are increasingly adopted to improve manufacturing flexibility and long-term operational efficiency.

The proposed solution additionally demonstrates ethical and sustainability awareness by reducing repetitive manual labour, improving operator safety and supporting safer collaborative operation through sensor-based monitoring and speed reduction strategies.

7. Performance Evaluation and Limitations

7.1. Cycle Time and Throughput Evaluation

A **five-minute** simulation test was performed at different robot speed settings to evaluate the throughput capability of the proposed robotic sorting workcell. The test measured the number of phone cases successfully detected, picked, sorted and placed within a 300-second operating period.

The results showed that the UR20 robot was capable of completing the primary pick-and-place operation between the conveyor and sorting box within approximately 2 seconds, satisfying the main client handling requirement. However, the overall production cycle increased due to the additional return motion required for the robot to travel back to the conveyor pickup position before processing the next phone case.

The return trajectory therefore became the primary factor limiting throughput during continuous operation rather than the pickup and placement task itself. In addition, the conveyor speed was intentionally limited to 0.05 m/s during testing to maintain stable sensing and repeatable pickup conditions. Further optimisation of conveyor speed and robot return trajectories could therefore improve overall throughput during extended production operation.

Robot Speed	Cases in 5 min	Average Cycle Time	Meets 2 s Target?
0.5 m/s	77	3.90 s/case	No
0.75 m/s	81	3.70 s/case	No
1.0 m/s	100	3.00 s/case	No
5.0 m/s	128	2.34 s/case	Close

Table 12: Five-minute throughput test results for different robot speed settings

The simulation results demonstrated reliable continuous sorting performance while approaching the required industrial production throughput target. The implemented finite state machine controller, predefined motion paths and approach-point strategy enabled stable and repeatable operation during repeated simulation cycles.

7.2. System Limitations

Several limitations were identified during evaluation of the proposed workcell. The current simulation used simplified colour detection logic and did not include advanced machine-vision defect analysis. Conveyor speed was intentionally reduced to maintain stable pickup conditions, limiting overall throughput performance. The implemented robot trajectories additionally prioritised collision avoidance and repeatability rather than full cycle-time optimisation. Dynamic obstacle avoidance and predictive motion planning were also not implemented within the current controller structure.

7.3. Future Improvements

Future improvements could further enhance throughput, flexibility and industrial realism. Optimisation of robot return trajectories, reduced approach distances and smoother path interpolation could significantly reduce overall cycle time. Integration of advanced vision systems or AI-based inspection algorithms could improve defect detection capability and product quality control.

Additional safety features such as dynamic obstacle avoidance, predictive safety monitoring or automated mobile robot (AMR) integration for box transportation could further improve operational flexibility and scalability. Multi-robot coordination or dual-conveyor configurations could also be investigated to support higher production rates for large-scale manufacturing deployment.

References

- AMD Machines (n.d.) ISO 10218 & ISO/TS 15066 Explained: Robot Safety Standards for Integrators. Available at: <https://amdmachines.com/blog/robot-safety-standards-iso-10218-and-ts-15066-explained/> (Accessed: 22 May 2026).
- Department for Science, Innovation and Technology (2023) National Vision for Engineering Biology and Advanced Manufacturing. UK Government.
- IEC (2018) IEC 60204-1:2018 Safety of machinery – Electrical equipment of machines – Part 1: General requirements. Geneva: International Electrotechnical Commission.
- ISO (2016) ISO/TS 15066:2016 Robots and robotic devices – Collaborative robots. Geneva: International Organization for Standardization.
- ISO (2025a) ISO 10218-1:2025 Robotics – Safety requirements – Part 1: Industrial robots. Geneva: International Organization for Standardization. Available at: <https://www.iso.org/standard/73933.html> (Accessed: 22 May 2026).
- ISO (2025b) ISO 10218-2:2025 Robotics – Safety requirements – Part 2: Industrial robot applications and robot cells. Geneva: International Organization for Standardization. Available at: <https://www.iso.org/standard/73934.html> (Accessed: 22 May 2026).
- Make UK (2023) The Future of UK Manufacturing Report. Available at: <https://www.makeuk.org> (Accessed: 22 May 2026).
- MathWorks (2025) Inverse Kinematics Designer – MATLAB Documentation. Available at: <https://uk.mathworks.com/help/releases/R2025b/robotics/ref/inversekinematicsdesigner-app.html> (Accessed: 22 May 2026).
- Office for National Statistics (ONS) (2023) Automation and the Future Workforce in the UK. Available at: <https://www.ons.gov.uk> (Accessed: 22 May 2026).
- OnRobot (n.d.) VGC10 – Small but powerful electric vacuum gripper. Available at: <https://onrobot.com/en/products/vgc10-vacuum-gripper> (Accessed: 22 May 2026).
- Schwab, K. (2016) The Fourth Industrial Revolution. Geneva: World Economic Forum.
- techUK (2023) UK Robotics and Autonomous Systems Adoption Report. Available at: <https://www.techuk.org> (Accessed: 22 May 2026).
- Universal Robots (n.d.a) DH Parameters for Calculations of Kinematics and Dynamics. Available at: <https://www.universal-robots.com/articles/ur/application-installation/dh-parameters-for-calculations-of-kinematics-and-dynamics/> (Accessed: 22 May 2026).
- Universal Robots (n.d.b) Stop Categories. Available at: https://www.universal-robots.com/manuals/EN/HTML/SW5_19/Content/prod-usr-man/complianceUR5e/H_g5_sections/safetyFunctionsAndinterfaces/stop_categories.htm (Accessed: 22 May 2026).

Appendix A

A.1 Comparative Universal Robots Specifications

The manufacturer specification sheets below were used to compare payload capacity, reach and workspace suitability during robot selection for the proposed Phones-R-Us automation system.

UNIVERSAL ROBOTS

A Tecumseh Products Company

April 2026

e-Series

Robot Arm Specification



UR3e



UR7e



UR12e



UR16e

Payload	3 kg (6.6 lbs)		7.5 kg (16.5 lbs)		12.5 kg (27.5 lbs)		16 kg (35.3 lbs)	
Reach	500 mm (19.7 in)		850 mm (33.5 in)		1300 mm (51.2 in)		900 mm (35.4 in)	
Degrees of freedom	6 rotating joints							
Programming	12 inch touchscreen with PolyScope graphical user interface							
Power consumption (average)								
Maximum power	300 W		570 W		615 W		585 W	
Moderate operating settings	150 W		200 W		350 W		350 W	
Operating temperature range	Ambient temperature: 0-50 °C (32-122 °F)							
Safety functions	21 configurable safety functions							
In compliance with	EN ISO 13849-1 (PLd category 3) and EN ISO 10218-1							

Performance

Force sensing, tool flange/torque sensor	Force, x-y-z	Torque, x-y-z	Force, x-y-z	Torque, x-y-z	Force, x-y-z	Torque, x-y-z	Force, x-y-z	Torque, x-y-z
Range	± 30.0 N	± 10.0 Nm	± 50.0 N	± 10.0 Nm	± 100.0 N	± 10.0 Nm	± 160.0 N	± 10.0 Nm
Precision	± 2.0 N	± 0.1 Nm	± 3.5 N	± 0.2 Nm	± 5.0 N	± 0.2 Nm	± 5.0 N	± 0.2 Nm
Accuracy	± 3.5 N	± 0.1 Nm	± 4.0 N	± 0.3 Nm	± 5.5 N	± 0.5 Nm	± 5.5 N	± 0.5 Nm

Movement

Maximum TCP speed	3 m/s		4 m/s		4 m/s		3 m/s	
Pose Repeatability per ISO 9283	± 0.03 mm		± 0.03 mm		± 0.05 mm		± 0.05 mm	
Axis movement	Working range	Maximum speed	Working range	Maximum speed	Working range	Maximum speed	Working range	Maximum speed
Base	± 360°	± 180°/s	± 360°	± 180°/s	± 360°	± 120°/s	± 360°	± 120°/s
Shoulder	± 360°	± 180°/s	± 360°	± 180°/s	± 360°	± 120°/s	± 360°	± 120°/s
Elbow	± 360°	± 180°/s	± 360°	± 180°/s	± 360°	± 180°/s	± 360°	± 180°/s
Wrist 1	± 360°	± 360°/s	± 360°	± 180°/s	± 360°	± 180°/s	± 360°	± 180°/s
Wrist 2	± 360°	± 360°/s	± 360°	± 180°/s	± 360°	± 180°/s	± 360°	± 180°/s
Wrist 3	Infinite	± 360°/s	± 360°	± 180°/s	± 360°	± 180°/s	± 360°	± 180°/s

Features

IP Classification	IP54							
Noise	< 60 dB(A)		< 65 dB(A)		< 65 dB(A)		< 65 dB(A)	
Arm mounting	Any orientation							
I/O Ports								
Digital In	2		2		2		2	
Digital Out	2		2		2		2	
Analog In	2		2		2		2	
Tool I/O power supply voltage	12/24V							
Tool I/O power supply	600 mA		1.5 A (Dual pin) 1 A (Single pin)		2 A (Dual pin) 1 A (Single pin)		2 A (Dual pin) 1 A (Single pin)	

Physical

Footprint	Ø128 mm	Ø151 mm	<Ø190 mm	Ø190 mm
Materials	Aluminum, plastic, steel			
Tool flange interface	M8 8-pin female, EN ISO 9409-1:50-4-M8			
Connector type	M8 8-pin female			
Cable length (robot arm)	6 m (236 in)			
Weight including cable	11.2 kg (24.7 lbs)	20.6 kg (45.4 lbs)	33.5 kg (73.9 lbs)	33.1 kg (73 lbs)
Humidity	≤ 90% RH (non-condensing)			

e-Series - Technical Specifications

e-Series - Technical Specifications

Figure 24: Universal Robots e-Series technical specification comparison used during robot evaluation





UNIVERSAL ROBOTS
Robot Technology

UR20

General specification

Payload	20 kg (44.1 lbs) 25 kg (55.1 lbs) with boundary conditions according to manual.
Reach	1750 mm (88.9 in)
Degrees of freedom	6 rotating joints
Programming	12 inch touchscreen with PolyScope graphical user interface
Power consumption (average)	
Maximum power	750 W
Moderate operating settings	500 W
Operating temperature range	Ambient temperature: 0-50 °C (32-122 °F)
Safety functions	21 configurable safety functions EN ISO 13849-1 (PLD Category 3), EN ISO 10218-1 and UL 1740
In compliance with	
Joint architecture	2+1-3

UR20 Robot Arm Performance

Force sensing, tool flange/torque sensor	Force, x-y-z	Torque, x-y-z
Range	± 200.0 N	± 20.0 Nm
Precision	± 5.5 N	± 0.2 Nm
Accuracy	± 10.0 N	± 0.1 Nm

Movement

Maximum TCP speed	5 m/s
Pose repeatability per ISO 9283	± 0.1 mm
Axis movement	Working range Maximum speed
Base	± 360° ± 120°/s
Shoulder	± 360° ± 120°/s
Elbow	± 360° ± 150°/s
Wrist 1	± 360° ± 210°/s
Wrist 2	± 360° ± 210°/s
Wrist 3	± 360° ± 210°/s

UR20

Technical specification

The UR20, the first of our next-generation high performance cobots, has been redesigned from the ground up to provide up to 30% more speed and torque, advanced motion control capabilities, and first-class user experience.

Today, more than 100,000 UR collaborative industrial robots have been delivered to customers across industries and around the world. The newest series of heavy payload cobots brings unparalleled performance, adaptability and an unparalleled user experience.

Features

IP Classification	IP65
Cleanroom classification	Class 6 at ≤ 80%*
ISO 14644-1 "Velocity and payload"	
Noise	< 65 dB(A)
Status identification	Light ring at robot base
Arm mounting	Any orientation
I/O ports at tool flange	
Digital In	2
Digital Out	2
Analog In	2 or 1 RS-485
Power supply voltage	12/24 V
Power supply	2 A (Dual pin), 1 A (Single pin)

Physical

Footprint	Ø245 mm
Materials	Aluminum, plastic, steel
Tool flange	EN ISO 9409-1-80-6-M8
Connector type	M8 8-pin female
Cable length (robot arm)	6 m (236 in)
Weight without cable	64 kg (141.1 lbs)
Humidity	≤ 90% RH (non-condensing)

3-Position Enabling Teach Pendant

Features

IP classification	IP54
Humidity	≤ 90% RH (non-condensing)
Display resolution	1280 x 800 pixels

Physical

Materials	Plastic
Teach pendant size (W x H x D)	300 mm x 231 mm x 50 mm (11.8 in x 9.1 in x 1.97 in)
Weight	1.8 kg (3.961 lbs) including 1 m of teach pendant cable
Cable length	4.5 m (177.17 in)

Control Box

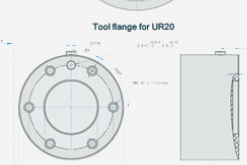
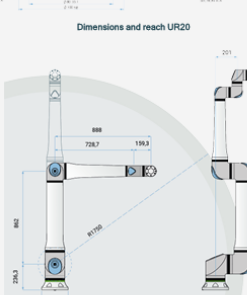
Features

IP classification	IP44
Humidity	≤ 90% RH (non-condensing)
I/O Ports	
Digital In	16
Digital Out	16
Analog In	2
Analog Out	2
Quadrature digital inputs	4
I/O Power Supply	24V, 2A
Industrial protocols	Modbus TCP (Client/Server) Ethernet/IP Adapter PROFINET device, PROFINET IRT/RS485 Ethernet 1 Gb/s USB 2.0 USB 3.0 Mini DisplayPort Injection Moulding Machine Interface (SPR AN 146 & Europap 377)

Hardware interfaces

Power Source
*Only valid for OEM version.



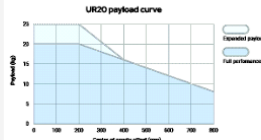
888
708.7
198.3
1750
245
201

Contact

Universal Robots A/S
Enervej 51
5260 Odense
Denmark

+45 89 93 89 89
sales@universal-robots.com
universal-robots.com

UR20 payload curve



Center of gravity offset (mm)

Payload (kg)

Legend: — Extended payload — Full performance

April 2026
UNLESS OTHERWISE SPECIFIED:
Dimensions are in millimeters. Tolerance ± 0.1 mm ± 0.5

39

The UR30 robot was evaluated as an alternative high-payload solution during the robot selection process. Although technically feasible, the robot was not selected due to unnecessary payload capacity and reduced economic justification for the proposed application.

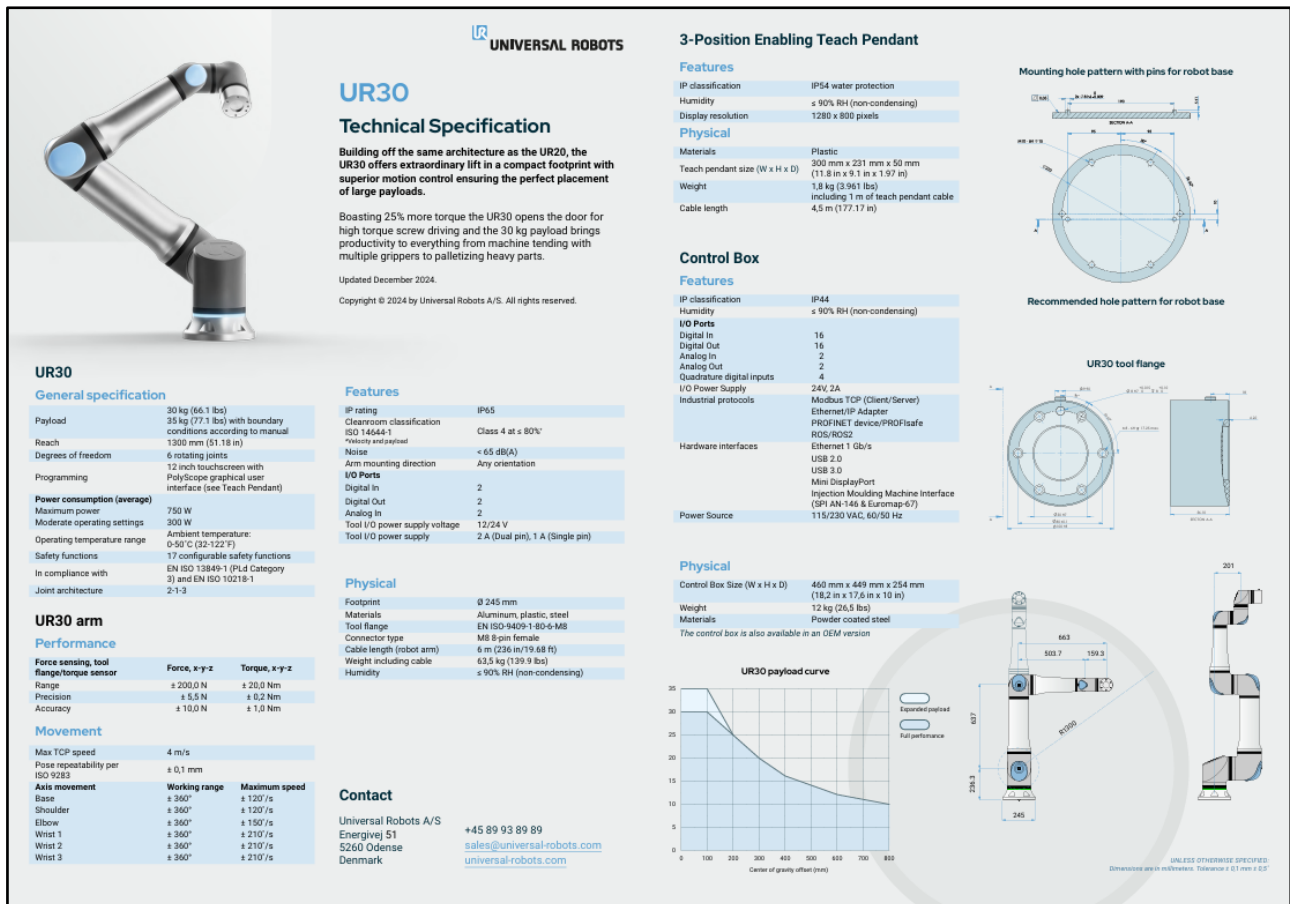


Figure 26: Manufacturer technical specification sheet for the UR30 collaborative robot evaluated during selection

Appendix B

B.4 Box Capacity Monitoring and Replacement

Figure 27 shows the implemented box filling and transportation operations coordinated by the robot main control script during continuous sorting execution.

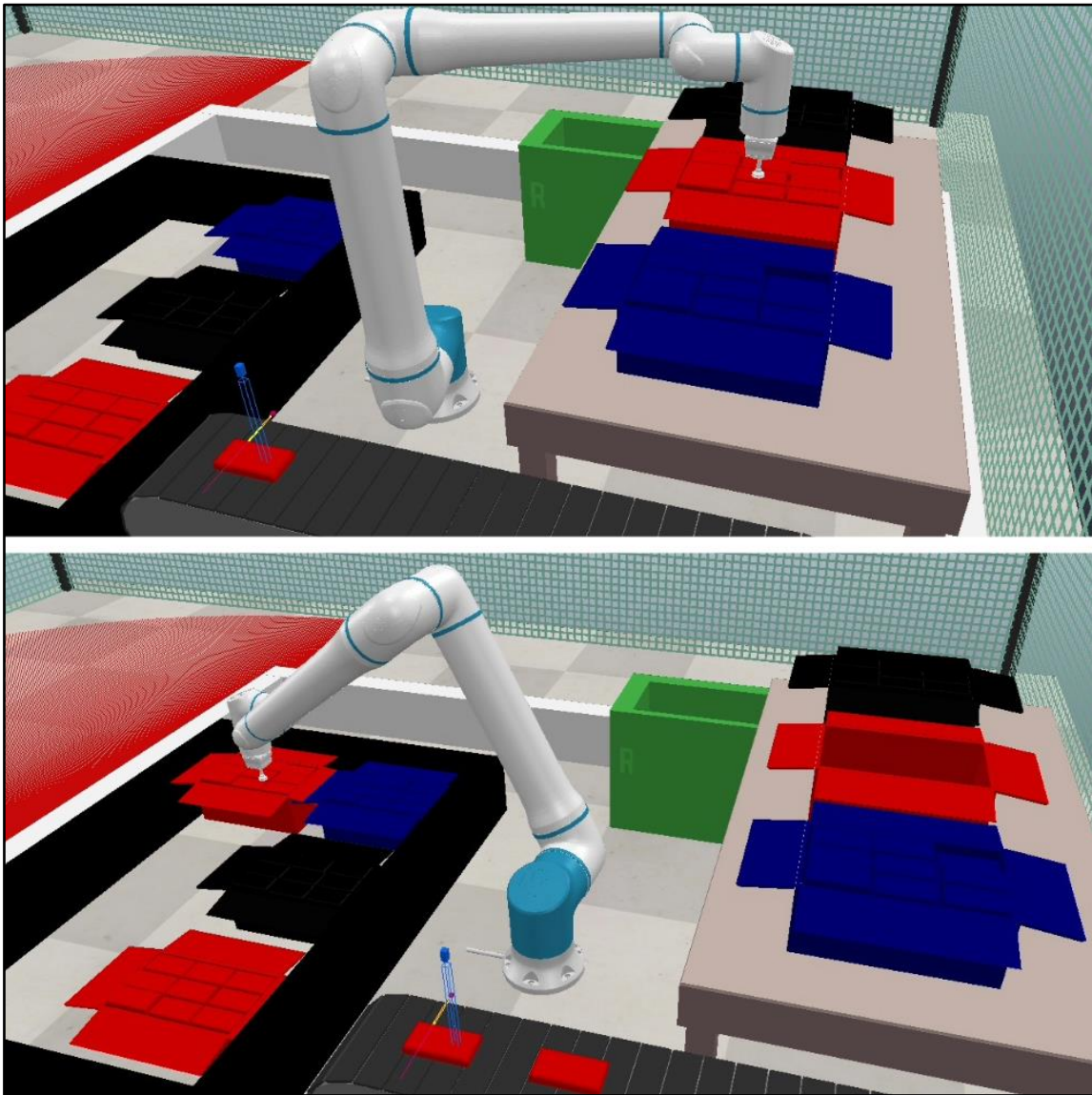


Figure 27: Automatic box filling and transportation operations implemented within the robot control system

B.3 Sorting and Placement Logic

The robot main script coordinated predefined placement positions to maintain organised stacking behaviour inside each sorting box.

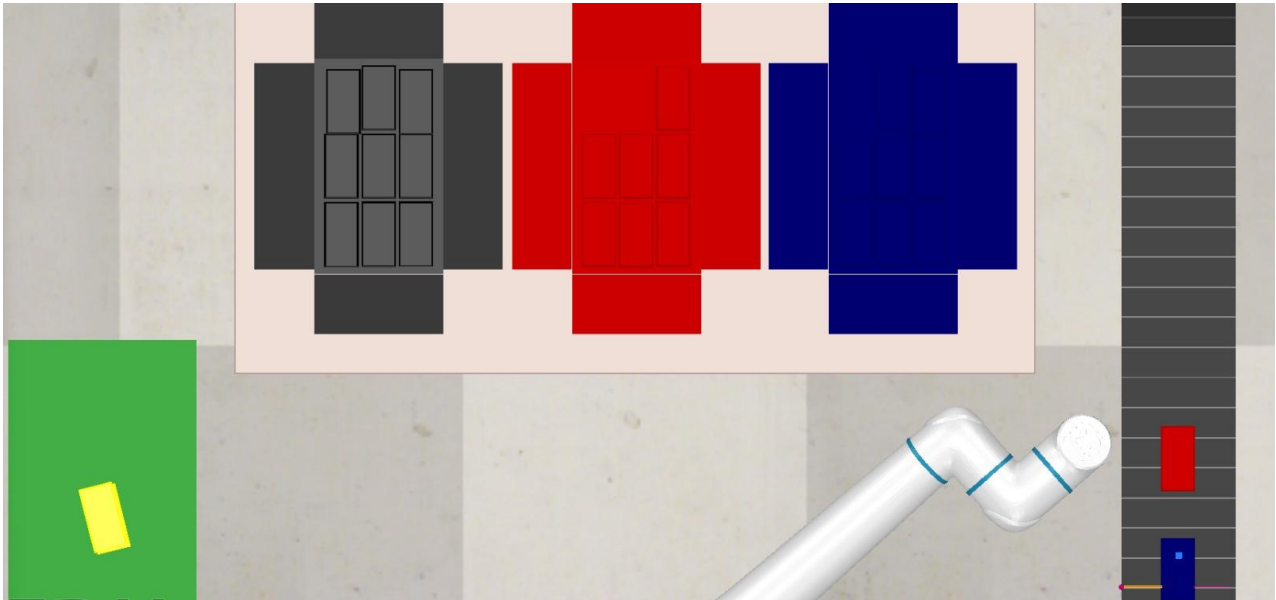


Figure 28: Top-view arrangement of phone case placement positions generated by the robot control logic

Appendix C

Figure 29 illustrates the colour classification sequence implemented within the vision processing subsystem.

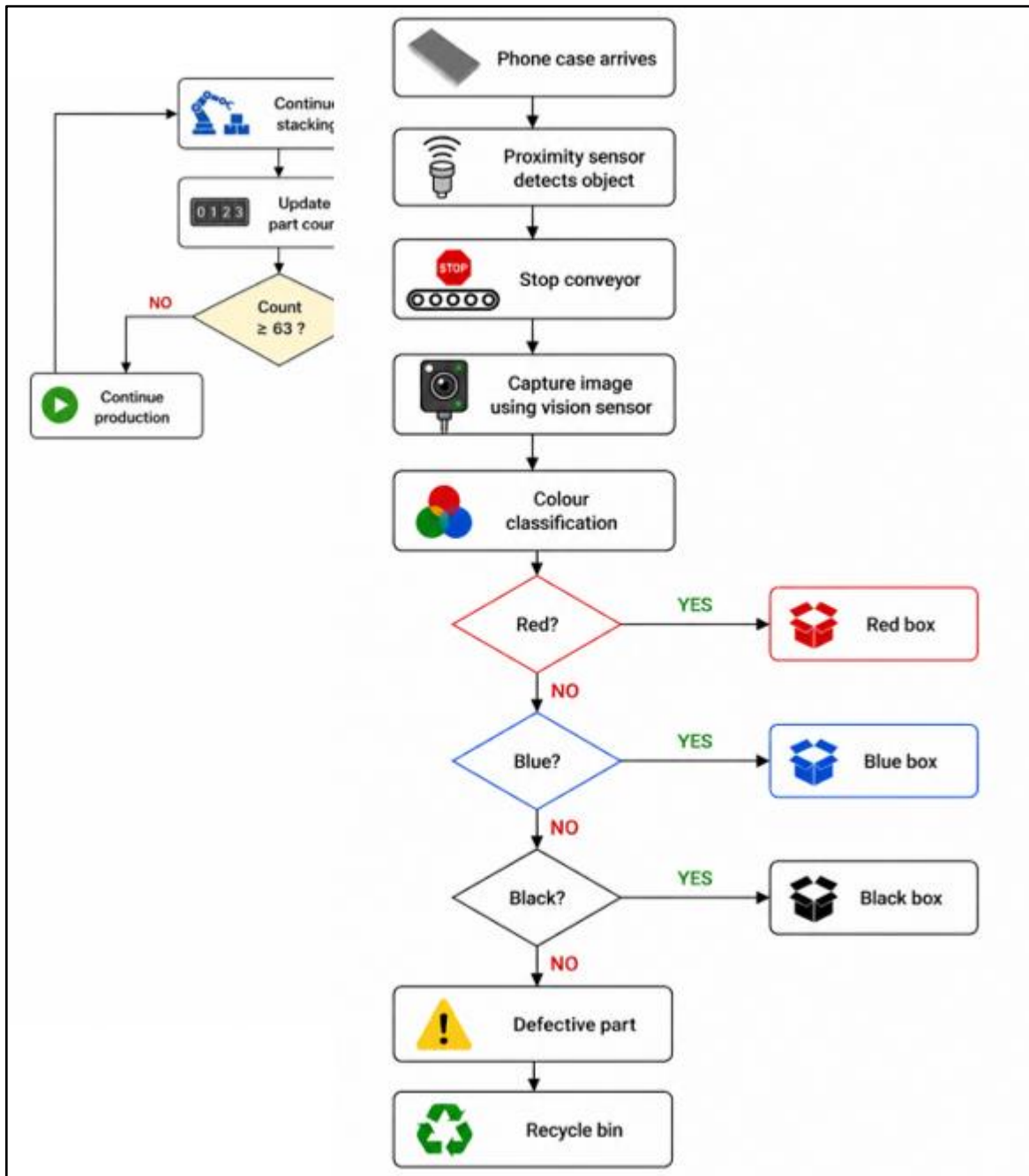


Figure 29: Vision sensor colour classification and sorting logic

Figure 30 shows the automatic box replacement sequence coordinated by the robot main control script during continuous production operation.

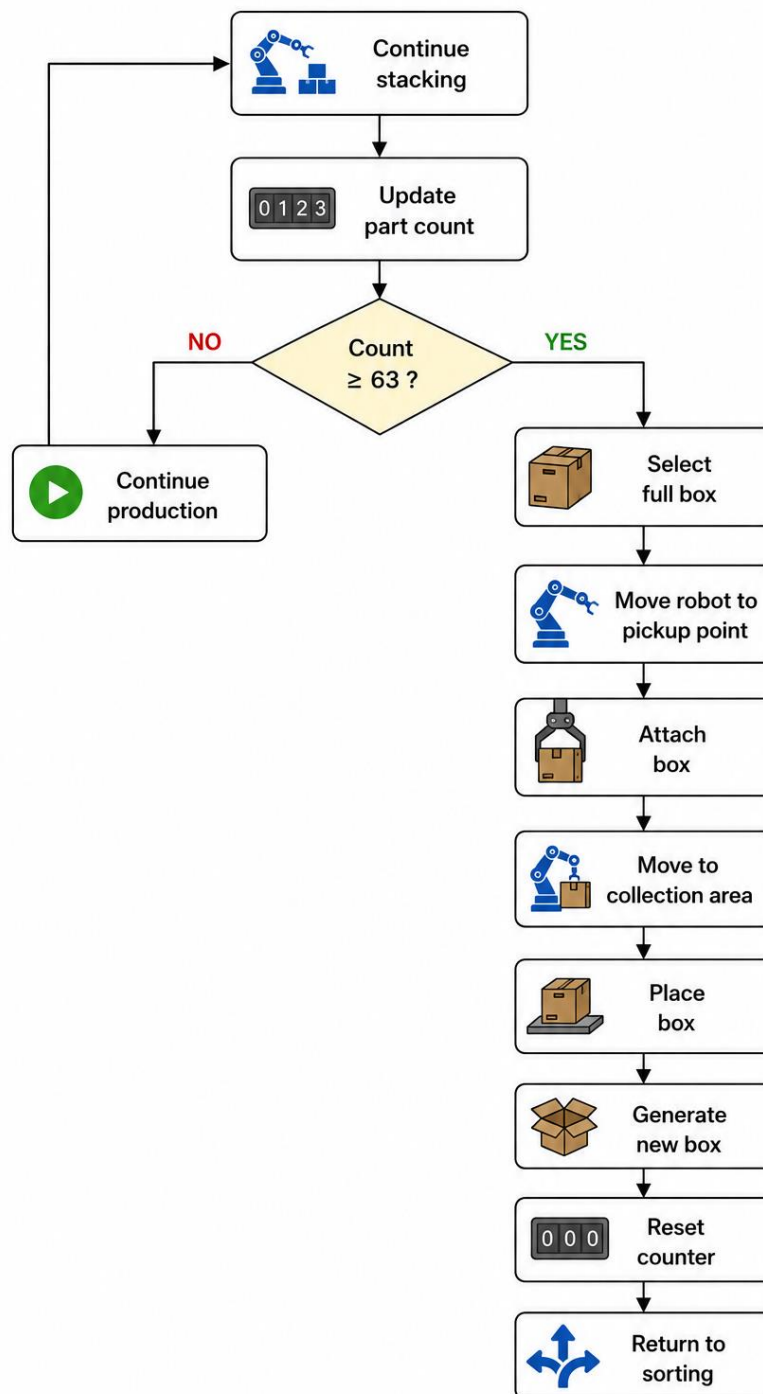


Figure 30: Automatic box handling and replacement sequence implemented within the robot control system

Appendix D

D.1 MATLAB Setup Parameters

The following supplementary MATLAB environment parameters were used to reconstruct the UR20 industrial sorting workcell and support the kinematic analysis performed throughout the project.

Parameter	Value
Robot Model	UR20
Degrees of Freedom	6
MATLAB Toolbox	Robotics System Toolbox
Robot Import Method	URDF Import
Workspace Size	3 × 3 m
Base Frame Reference	Robot Base at Origin
Simulation Reference	CoppeliaSim Workcell

Table 13: MATLAB environment and UR20 robot setup parameters used for industrial workcell reconstruction

D.2 Collision Object Definitions

Collision objects were created within MATLAB to represent the industrial workcell components used during workspace and kinematic validation.

Collision Object	Type	Dimensions (m)	Purpose
Conveyor	collisionBox	0.2 × 0.2 × 1.2	Phone case transport
Red Box	collisionBox	0.3 × 0.4336 × 0.1501	Red sorting
Blue Box	collisionBox	0.3 × 0.4336 × 0.1501	Blue sorting
Black Box	collisionBox	0.3 × 0.4336 × 0.1501	Black sorting
Recycle Bin	collisionBox	0.33 × 0.525 × 0.6	Defect handling
Collection Area	collisionBox	1.7 × 1.2 × 0.25	Full box storage
Table Surface	collisionBox	1.395 × 0.795 × 0.1	Workstation
Floor	collisionBox	6 × 6 × 0.24	Workspace floor

Table 14: Collision objects and workspace component definitions implemented within the MATLAB industrial environment

D.3 Complete Pose Configurations

Validated industrial robot poses were generated within MATLAB IK Designer to support pickup, sorting, recycle handling and collection operations.

Pose No.	Pose Name	Purpose
1	homePosition	Initial robot position
2	approachPickingPoint	Approach above conveyor
3	pickingPoint	Phone case pickup
4	approachBlueBox	Approach blue box
5	placeBluePhoneCase	Place blue phone case
6	approachRedBox	Approach red box
7	placeRedPhoneCase	Place red phone case
8	approachBlackBox	Approach black box
9	placeBlackPhoneCase	Place black phone case
10	approachRecycleBin	Approach recycle bin
11	placeDefectedPhoneCase	Place defective phone case
12	pickBlueBox	Pickup full blue box
13	pickRedBox	Pickup full red box
14	pickBlackBox	Pickup full black box
15	approachCollectionArea	Approach collection area
16	placeBox	Place full box

Table 15: Robot task poses defined using MATLAB IK Designer for industrial handling operations

Appendix E — Forward Kinematics Outputs

E.1 Forward Kinematics TCP Position Data

Forward kinematic analysis was used to calculate the TCP positions for all validated industrial task poses relative to the UR20 robot base frame.

Pose	TCP_X_m	TCP_Y_m	TCP_Z_m
"homePosition"	-0.20885	-0.95676	1.218
"approachPickingPoint"	-0.94966	-0.33032	0.96973
"pickingPoint"	-0.94915	-0.32635	0.72155
"approachBlueBox"	-0.51365	-0.91482	0.95633
"placeBluePhoneCase"	-0.52185	-0.92428	0.80323
"approachRedBox"	-0.065231	-0.91434	0.96278
"placeRedPhoneCase"	-0.066981	-0.92669	0.80651
"approachBlackBox"	0.38461	-0.91066	0.95843
"placeBlackPhoneCase"	0.38818	-0.92206	0.81098
"approachRecycleBin"	0.92072	-0.4378	0.96152
"placeDefectedPhoneCase"	0.93553	-0.44787	0.60443
"pickBlueBox"	-0.45742	-1.0307	0.85451
"pickRedBox"	-0.0020825	-1.0305	0.85289
"pickBlackBox"	0.44208	-1.0302	0.85307
"approachCollectionArea"	0.42131	0.57589	0.86575
"placeBox"	0.41611	0.57121	0.25257

Figure 31: TCP coordinates generated using forward kinematic analysis for validated industrial task poses

Appendix F

F.1 Complete Joint Configuration Data

Complete joint-space configurations generated for validated industrial task poses are provided within this appendix for supplementary kinematic reference.

	J1	J2	J3	J4	J5	J6
homePosition	89.619	-98.299	-59.029	-111.92	89.447	179.97
approachPickingPoint	30.825	-96.599	-80.806	266.84	-270.73	287.23
pickingPoint	30.53	-98.526	-98.384	286.56	-270.02	286.94
approachBlueBox	71.745	-99.345	-78.952	269.06	-270.09	161.1
placeBluePhoneCase	71.419	-101.18	-88.453	278.73	-269.68	161.71
approachRedBox	98.659	-90.306	-88.357	269.3	-270.42	188.01
placeRedPhoneCase	98.297	-92.281	-98.337	279.7	-269.63	188.39
approachBlackBox	-235.36	-95.155	-83.648	269.76	-270.08	-145.09
placeBlackPhoneCase	124.41	-97.019	-92.768	278.81	-269.94	214.58
approachRecycleBin	-194.04	-97.272	-81.024	269.29	-270.14	-104.05
placeDefectedPhoneCase	-194.44	-103.61	-101.81	294.45	-269.79	-104.62
pickBlueBox	76.284	-104.96	-79.942	275.61	-269.63	166.89
pickRedBox	101.14	-98.336	-88.347	277.68	-270.03	191.23
pickBlackBox	-236.44	-104.46	-80.766	276.22	-270.04	-146.3
approachCollectionArea	-109.83	-76.55	-107.68	273.91	-269.99	-20.777
placeBox	-109.55	-104.46	-140.75	334.26	-270	-19.223

Figure 32: Joint-space configurations for validated UR20 industrial task poses

Appendix G

G.1 Supplementary Workspace Validation

Supplementary workspace representations were generated to further validate the spatial arrangement and accessibility of industrial task locations within the UR20 robotic workcell.

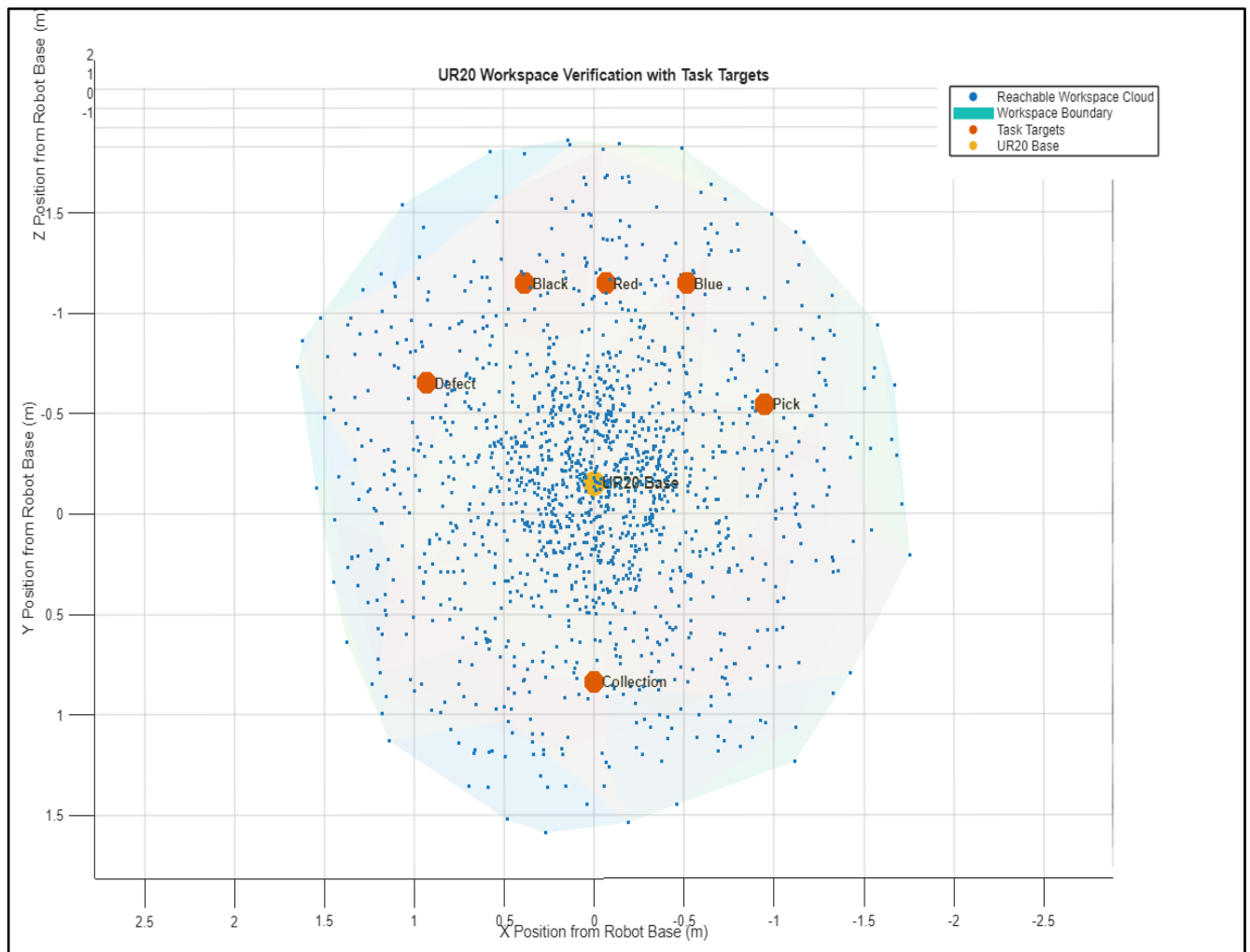


Figure 33: Top-view workspace validation of industrial task locations within the UR20 workcell

The supplementary top-view representation further validates the horizontal workstation arrangement between conveyor pickup, sorting and collection operations.

Appendix H

ROBOT SCRIPT

--

=====

=====

-- UR20 PICK AND PLACE AUTOMATION CELL

--

-- Application:

-- Automated sorting of mobile phone cases using colour

-- identification and robotic handling.

--

-- Robot Tasks:

-- 1. Pick phone case from conveyor

-- 2. Sort red, blue and black parts

-- 3. Send defective parts to recycle bin

-- 4. Stack parts inside boxes

-- 5. Replace full boxes automatically

--

-- Additional Features:

-- ? Vision-based colour classification

-- ? Safety scanner speed control

-- ? Collaborative operation mode

-- ? Defective product handling

-- ? Automatic box transport

--

-- Main control method:

-- Finite State Machine (FSM)

--

=====

=====

sim = require('sim')


```

simIK = require('simIK')

function sysCall_init()

    --
    =====
    =====

    -- ROBOT IK COMPONENTS
    -- simBase   : Robot base frame
    -- ikTip     : End-effector frame
    -- simTarget : IK target moved during motion planning

    --
    =====
    =====

    simBase = sim.getObject('..')
    ikTip = sim.getObject('..'/tip')
    simTarget = sim.getObject('/IkTarget')

    --
    =====
    =====

    -- PICKING LOCATIONS
    -- approachPick : Safe approach position above conveyor
    -- pickPoint    : Final grasp point

    --
    =====
    =====

    pickPoint = sim.getObject('/pickingPoint')
    approachPick = sim.getObject('/approachPickingPoint')

    --
    =====
    =====

```

-- SORTING DESTINATIONS

-- Red parts -> Red box

-- Blue parts -> Blue box

-- Black parts -> Black box

-- Other parts -> Recycling bin

--

=====

blackBox = sim.getObject('/blackPhoneBox')

redBox = sim.getObject('/redPhoneBox')

blueBox = sim.getObject('/bluePhoneBox')

recyclingBin = sim.getObject('/recyclingBin')

approachBlackBox = sim.getObject('/approachBlackBox')

approachRedBox = sim.getObject('/approachRedBox')

approachBlueBox = sim.getObject('/approachBlueBox')

approachRecycleBin = sim.getObject('/approachRecycleBin')

placeBlackPhone = sim.getObject('/placeBlackPhone')

placeRedPhone = sim.getObject('/placeRedPhone')

placeBluePhone = sim.getObject('/placeBluePhone')

placeYellowPhone = sim.getObject('/placeYellowPhone')

pickBlackBox = sim.getObject('/pickBlackBox')

pickRedBox = sim.getObject('/pickRedBox')

pickBlueBox = sim.getObject('/pickBlueBox')

approachCollectionArea = sim.getObject('/approachCollectionArea')

placeBox = sim.getObject('/placeBox')

```

blueBoxPathHandle = sim.getObject('/blueBoxPath')
redBoxPathHandle  = sim.getObject('/redBoxPath')
blackBoxPathHandle = sim.getObject('/blackBoxPath')

```

```
--
```

```
=====
=====
```

```
-- BOX STACKING GRID
```

```
--
```

```
-- Parts are arranged in:
```

```
-- 3 ? 3 arrangement per layer
```

```
-- Multiple vertical layers
```

```
--
```

```
-- xSpacing : distance between columns
```

```
-- ySpacing : distance between rows
```

```
-- zSpacing : layer height increment
```

```
--
```

```
=====
=====
```

```
xSpacing = 0.065
```

```
ySpacing = -0.119
```

```
zSpacing = 0.021
```

```
countRed, countBlue, countBlack, countYellow = 0,0,0,0
```

```
--
```

```
=====
=====
```

```
-- FINITE STATE MACHINE VARIABLES
```

```
--
```

```
-- waiting
```

```

-- follow_path
-- move_to_pick
-- attach_object
-- move_to_box
-- place_object
-- detach_object
-- follow_box_path
-- place_box
--

-- objectToPick stores current workpiece handle
--

=====
=====

state = "waiting"
objectToPick = -1

currentTargetApproach = nil
currentTargetPlace = nil

collectionCount = 0

targetSpeed = 0.5
boxToHandle = nil

--

=====
=====

-- SAFETY SYSTEM

--

-- Distance sensor zones:

--

```

```

-- SAFE:
-- Robot runs at full speed
--
-- WARNING:
-- Robot speed reduced
--
-- DANGER:
-- Emergency stop activated
--
-- Safety signal obtained from scanner
--

```

```

=====
=====

```

```

robotSpeedFactor = 1.0
dangerDist = 0.5
warningDist = 2.0

```

```

uiSpeedFactor = 1.0

```

```

--
=====
=====

```

```

-- INITIAL PICK PATH
--
-- Robot follows predefined trajectory from home position toward conveyor region.
-- Path points are extracted from custom path object.
--

```

```

=====
=====

```

```

pathHandle = sim.getObject('/Path')

```

```

local pathData = sim.unpackDoubleTable(
    sim.getBufferProperty(pathHandle, 'customData.PATH')
)

```

```

local m = Matrix(#pathData // 7, 7, pathData)
pathPositions = m:slice(1,1,m:rows(),3):data()

```

```

pathIndex = 1
pathAlreadyExecuted = false

```

```

--

```

```

=====
=====

```

```

-- DEFECTIVE PRODUCT PATH

```

```

--

```

```

-- Used when colour is different from blue, red and black
-- Robot moves directly toward recycle bin

```

```

--

```

```

=====
=====

```

```

defectPathHandle = sim.getObject('/defectedPartPath')

```

```

local defectPathData = sim.unpackDoubleTable(
    sim.getBufferProperty(defectPathHandle, 'customData.PATH')
)

```

```

local m2 = Matrix(#defectPathData // 7, 7, defectPathData)
defectPathPositions = m2:slice(1,1,m2:rows(),3):data()

```

```

defectPathParam = 0

```



```
defectPathIndex = 1
```

```
followDefectPath = false
```

```
--
```

```
=====
```

```
-- LOAD PREDEFINED BOX TRANSPORT PATHS
```

```
-- These paths move full boxes from sorting area
```

```
-- to the collection/storage location
```

```
--
```

```
=====
```

```
bluePathPositions = loadPathData(blueBoxPathHandle)
```

```
redPathPositions  = loadPathData(redBoxPathHandle)
```

```
blackPathPositions = loadPathData(blackBoxPathHandle)
```

```
-- Active path information used during box transport
```

```
boxPathPositions = {}
```

```
boxPathIndex = 1
```

```
followBoxPath = false
```

```
--
```

```
=====
```

```
-- LOAD RETURN PATH FOR DEFECTIVE OBJECT HANDLING
```

```
-- Robot returns to normal operation after placing
```

```
-- defective products into recycling area
```

```
--
```

```
=====
```

```
defectReturnPathHandle = sim.getObject('/defectedPartReturnPath')
```

```

defectReturnPathPositions = loadPathData(defectReturnPathHandle)

defectReturnPathIndex = 1
followDefectReturnPath = false

--
=====
=====

-- CREATE INVERSE KINEMATICS ENVIRONMENT
-- Used to control robot motion through target poses
--
=====
=====

ikEnv = simIK.createEnvironment()

--
=====
=====

-- PRIMARY IK SOLVER
-- Uses pseudo-inverse method for normal robot motion
--
=====
=====

ikGroup_undamped = simIK.createGroup(ikEnv)
simIK.setGroupCalculation(ikEnv, ikGroup_undamped, simIK.method_pseudo_inverse, 0, 6)
simIK.addElementFromScene(ikEnv, ikGroup_undamped, simBase, ikTip, simTarget,
simIK.constraint_pose)

```

```

--
=====

-- SECONDARY IK SOLVER

-- Uses damped least squares to improve convergence
-- near singularities or difficult poses

--
=====

ikGroup_damped = simIK.createGroup(ikEnv)

simIK.setGroupCalculation(ikEnv, ikGroup_damped, simIK.method_damped_least_squares,
0.3, 99)

simIK.addElementFromScene(ikEnv, ikGroup_damped, simBase, ikTip, simTarget,
simIK.constraint_pose)

end

function loadPathData(pathHandle)
    local data = sim.unpackDoubleTable(
        sim.getBufferProperty(pathHandle, 'customData.PATH')
    )
    local m = Matrix(#data // 7, 7, data)
    return m:slice(1,1,m:rows(),3):data()
end

-- ===== SAFETY FUNCTION =====

function updateSafetyScanner()

    local dist = sim.getFloatSignal("safety_distance")

    if not dist then
        robotSpeedFactor = 1.0
    end
end

```

```

        return
    end

    if dist < dangerDist then
        robotSpeedFactor = 0.0
    elseif dist < warningDist then
        robotSpeedFactor = 0.2
    else
        robotSpeedFactor = 1.0
    end
end

--
=====
=====

-- MOTION INTERPOLATION FUNCTION
--
-- Moves IK target gradually toward goal position.
-- Motion speed affected by:
--
-- -> Robot nominal speed
-- -> Safety reduction factor
-- -> UI speed setting
--
-- Prevents sudden jumps.
--
=====
=====

function stepToward(goalPos, goalOri)
    local dt = sim.getSimulationTimeStep()

```

-- SAFETY SPEED SCALING

local maxStep = targetSpeed * robotSpeedFactor * uiSpeedFactor * dt

local currentPos = sim.getObjectPosition(simTarget, -1)

local dx = goalPos[1] - currentPos[1]

local dy = goalPos[2] - currentPos[2]

local dz = goalPos[3] - currentPos[3]

local dist = math.sqrt(dx*dx + dy*dy + dz*dz)

if dist < maxStep then

 sim.setObjectPosition(simTarget, -1, goalPos)

 sim.setObjectOrientation(simTarget, -1, goalOri)

else

 local scale = maxStep / dist

 sim.setObjectPosition(simTarget, -1, {

 currentPos[1] + dx*scale,

 currentPos[2] + dy*scale,

 currentPos[3] + dz*scale

 })

end

end

--

=====

-- INVERSE KINEMATIC SOLVER

--

-- First attempt:

-- Pseudo inverse solution

```

--
-- Backup:
-- Damped least squares method
--
-- Improves stability near singularities
--
=====

function solveIK()
    if simIK.handleGroup(ikEnv, ikGroup_undamped, {syncWorlds=true}) ~=
simIK.result_success then
        simIK.handleGroup(ikEnv, ikGroup_damped, {syncWorlds=true})
    end
end

function reached(goalPos)
    local tipPos = sim.getObjectPosition(ikTip, -1)
    return math.sqrt(
        (tipPos[1]-goalPos[1])^2 +
        (tipPos[2]-goalPos[2])^2 +
        (tipPos[3]-goalPos[3])^2
    ) < 0.01
end

--
=====

-- BOX POSITION CALCULATOR
--
-- Calculates placement coordinates for stacked phone cases.
--

```

-- Returns:

-- x offset

-- y offset

-- z offset

--

=====

function getPlacementOffset(count)

 local layer = math.floor(count / 9)

 local index = count % 9

 local x = (index % 3) * xSpacing

 local y = math.floor(index / 3) * ySpacing

 local z = layer * zSpacing

 return x,y,z

end

--

=====

-- MAIN ROBOT CONTROL LOOP

--

-- This function executes the complete pick-and-place cycle:

-- 1. Read operating mode and safety status

-- 2. Detect and classify phone cases

-- 3. Execute pick operation

-- 4. Sort objects according to colour

-- 5. Handle defective products

-- 6. Replace full boxes

-- 7. Update production counters


```

--
=====

=====

function sysCall_actuation()

    --
    =====
    =====

    -- READ OPERATION MODE AND SPEED SETTINGS
    -- Normal mode : User speed control
    -- Collaborative: Fixed reduced speed

    --
    =====
    =====

    local mode = sim.getInt32Signal("mode")
    local uiSpeed = sim.getFloatSignal("ui_speed_factor") or 1.0

    if mode == 1 then
        -- Collaborative Mode (OVERRIDE everything)
        uiSpeedFactor = 0.3
    else
        -- Normal Mode (UI controlled)
        uiSpeedFactor = uiSpeed
    end

    sim.getFloatSignal("safety_distance")

    --
    =====
    =====

    -- SAFETY MONITORING

```

```

-- Stop or slow robot according to scanner distance
--
=====

=====

updateSafetyScanner()

-- EMERGENCY STOP
if robotSpeedFactor == 0 then
    return
end

--
=====

=====

-- UPDATE NEWLY GENERATED BOX HANDLES
-- Used after full box replacement
--
=====

=====

local newRed = sim.getInt32Signal("new_red_box")
if newRed and newRed ~= 0 then redBox = newRed sim.setInt32Signal("new_red_box",0)
end

local newBlue = sim.getInt32Signal("new_blue_box")
if newBlue and newBlue ~= 0 then blueBox = newBlue
sim.setInt32Signal("new_blue_box",0) end

local newBlack = sim.getInt32Signal("new_black_box")
if newBlack and newBlack ~= 0 then blackBox = newBlack
sim.setInt32Signal("new_black_box",0) end

```

```

--
=====
=====

-- CHECK SYSTEM ENABLE STATUS

--
=====
=====

if sim.getInt32Signal("system_enable") == 0 then return end

--
=====
=====

-- STATE: WAITING
-- Wait for object detection and determine destination

--
=====
=====

if state=="waiting" then
    local spawned = sim.getInt32Signal("spawned_case_handle")
    local detected = sim.getInt32Signal("detectedBox")
    local handle = sim.getInt32Signal("detectedObjectHandle")

    followDefectPath = false -- reset every cycle
    followDefectReturnPath = false
    defectPathIndex = 1
    defectReturnPathIndex = 1

    if spawned~=0 and detected==1 then
        objectToPick = handle

        local color = sim.getStringSignal("detectedColor")

```

```

if color=="red" then
    currentTargetApproach=approachRedBox
    currentTargetPlace=placeRedPhone

elseif color=="blue" then
    currentTargetApproach=approachBlueBox
    currentTargetPlace=placeBluePhone

elseif color=="black" then
    currentTargetApproach=approachBlackBox
    currentTargetPlace=placeBlackPhone

else
    currentTargetApproach=approachRecycleBin
    currentTargetPlace=placeYellowPhone
    followDefectPath = true
end

sim.setInt32Signal("spawned_case_handle",0)

-- ? KEY LOGIC HERE
if pathAlreadyExecuted == false then
    state = "follow_path"
else
    state = "move_to_approach"
end
end
end
end

```

```

--
=====
=====

-- STATE: FOLLOW INITIAL PATH
-- Move robot from home position to pick region

--
=====
=====

if state == "follow_path" then

    if #pathPositions == 0 then
        state = "move_to_approach"
        return
    end

    local goal = {
        pathPositions[3*(pathIndex-1)+1],
        pathPositions[3*(pathIndex-1)+2],
        pathPositions[3*(pathIndex-1)+3]
    }

    local ori = sim.getObjectOrientation(approachPick, -1)

    stepToward(goal, ori)
    solveIK()

    if reached(goal) then
        pathIndex = pathIndex + 1

        if pathIndex > (#pathPositions / 3) then
            pathIndex = 1

```

```

    pathAlreadyExecuted = true

    -- SMALL STABILITY FIX
    sim.setObjectPosition(simTarget, -1, goal)

    state = "move_to_approach"
end
end
end

--
=====
=====

-- STATE: FOLLOW DEFECT PATH
-- Move defective products toward recycle bin
--
=====
=====

if state=="follow_defect_path" then

    if #defectPathPositions == 0 then
        state = "move_to_approach"
        return
    end

    local goal = {
        defectPathPositions[3*(defectPathIndex-1)+1],
        defectPathPositions[3*(defectPathIndex-1)+2],
        defectPathPositions[3*(defectPathIndex-1)+3]
    }

```

```

local ori = sim.getObjectOrientation(approachRecycleBin, -1)

stepToward(goal, ori)
solveIK()

if reached(goal) then
    defectPathIndex = defectPathIndex + 1

    if defectPathIndex > (#defectPathPositions / 3) then
        defectPathIndex = 1
        followDefectPath = false
        state = "move_to_box"
    end
end
end

--
=====
=====

-- STATE: MOVE TO PICK APPROACH POSITION
-- Safe position above conveyor
--
=====
=====

if state=="move_to_approach" then
    local p,o = sim.getObjectPosition(approachPick,-1),
sim.getObjectOrientation(approachPick,-1)
    stepToward(p,o) solveIK()
    if reached(p) then state="move_to_pick" end
end
end

```

```

--
=====
=====

-- STATE: MOVE TO PICK POINT
-- Robot reaches object grasp position
--
=====
=====

if state=="move_to_pick" then
    local p,o = sim.getObjectPosition(pickPoint,-1), sim.getObjectOrientation(pickPoint,-1)
    stepToward(p,o) solveIK()
    if reached(p) then state="attach_object" end
end

--
=====
=====

-- STATE: ATTACH OBJECT
-- Parent object to robot end effector
--
=====
=====

if state=="attach_object" then
    sim.setObjectInt32Param(objectToPick, sim.shapeintparam_static,1)
    sim.setObjectInt32Param(objectToPick, sim.shapeintparam_respondable,0)
    sim.setObjectParent(objectToPick, ikTip, true)

-- ?? KEY LOGIC
if followDefectPath == true then
    state = "follow_defect_path"
else

```



```

        state = "move_up"
    end
end

--
=====
=====

-- STATE: MOVE UP
-- Lift object safely from conveyor
--
=====
=====

if state=="move_up" then
    local p,o = sim.getObjectPosition(approachPick,-1),
sim.getObjectOrientation(approachPick,-1)
    stepToward(p,o) solveIK()
    if reached(p) then state="move_to_box" end
end

--
=====
=====

-- STATE: MOVE TO DESTINATION BOX
-- Select destination using detected colour
--
=====
=====

if state=="move_to_box" then
    local p,o = sim.getObjectPosition(currentTargetApproach,-1),
sim.getObjectOrientation(currentTargetApproach,-1)
    stepToward(p,o) solveIK()

```

```

    if reached(p) then state="place_object" end
end

```

```

--

```

```

=====
=====

```

```

-- STATE: PLACE OBJECT
-- Calculate stacking position inside box

```

```

--

```

```

=====
=====

```

```

if state=="place_object" then
    local base = sim.getObjectPosition(currentTargetPlace,-1)
    local ori = sim.getObjectOrientation(currentTargetPlace,-1)

```

```

    local pos = base

```

```

if currentTargetPlace ~= placeYellowPhone then

```

```

    local count = (currentTargetPlace==placeRedPhone and countRed)
        or (currentTargetPlace==placeBluePhone and countBlue)
        or countBlack

```

```

    local dx,dy,dz = getPlacementOffset(count)
    pos = {base[1]+dx, base[2]+dy, base[3]+dz}
end

```

```

stepToward(pos,ori) solveIK()
if reached(pos) then state="detach_object" end
end

```

```
--
=====
=====

-- STATE: DETACH OBJECT

-- Release object into storage box or recycle bin

--
=====
=====
```

```
if state=="detach_object" then
    sim.setObjectInt32Param(objectToPick, sim.shapeintparam_respondable,1)

    if currentTargetPlace==placeRedPhone then
        sim.setObjectInt32Param(objectToPick, sim.shapeintparam_static,1)
        sim.setObjectParent(objectToPick, redBox,true)
        countRed=countRed+1

    elseif currentTargetPlace==placeBluePhone then
        sim.setObjectInt32Param(objectToPick, sim.shapeintparam_static,1)
        sim.setObjectParent(objectToPick, blueBox,true)
        countBlue=countBlue+1

    elseif currentTargetPlace==placeBlackPhone then
        sim.setObjectInt32Param(objectToPick, sim.shapeintparam_static,1)
        sim.setObjectParent(objectToPick, blackBox,true)
        countBlack=countBlack+1

    else
        sim.setObjectInt32Param(objectToPick, sim.shapeintparam_static,0)
        sim.setObjectParent(objectToPick, recyclingBin,true)
        countYellow=countYellow+1
        followDefectReturnPath = true
```

```

end

if followDefectReturnPath == true then
    state = "follow_defect_return_path"
else
    state = "move_up_after_place"
end
end

--
=====
=====

-- STATE: MOVE UP AFTER PLACING
-- Return to safe position
-- Check whether box is full
--
=====
=====

if state=="move_up_after_place" then
    local p,o = sim.getObjectPosition(currentTargetApproach,-1),
sim.getObjectOrientation(currentTargetApproach,-1)

    stepToward(p,o)
    solveIK()

    if reached(p) then
        currentTargetApproach = nil
        currentTargetPlace = nil
        objectToPick = -1

        -- CHECK IF ANY BOX IS FULL

```

```

    if countRed >= 63 or countBlue >= 63 or countBlack >= 63 then
        state = "check_box_state"
    else
        state = "waiting"
    end
end
end
end

```

```

--

```

```

=====
=====

```

```

-- STATE: CHECK BOX STATUS
-- Determine whether replacement is required

```

```

--

```

```

=====
=====

```

```

if state=="check_box_state" then
    if countRed>=63 then boxToHandle="red"
    elseif countBlue>=63 then boxToHandle="blue"
    elseif countBlack>=63 then boxToHandle="black"
    else state="waiting" return end

```

```

    state="move_to_picking_box"
end

```

```

--

```

```

=====
=====

```

```

-- STATE: MOVE TO FULL BOX
-- Move robot toward box pickup location

```

```
--
=====
=====
```

```
if state=="move_to_picking_box" then
    local target = (boxToHandle=="red" and pickRedBox)
        or (boxToHandle=="blue" and pickBlueBox)
        or pickBlackBox

    local p,o = sim.getObjectPosition(target,-1), sim.getObjectOrientation(target,-1)
    stepToward(p,o) solveIK()
    if reached(p) then state="attach_box" end
end
```

```
--
=====
=====
```

```
-- STATE: ATTACH BOX
-- Grasp full storage box
```

```
--
=====
=====
```

```
if state=="attach_box" then
    local box = (boxToHandle=="red" and redBox)
        or (boxToHandle=="blue" and blueBox)
        or blackBox

    sim.setObjectParent(box, ikTip, true)
    currentBoxHandle = box

    -- Select path based on box color
```

```

if boxToHandle == "red" then
    boxPathPositions = redPathPositions
elseif boxToHandle == "blue" then
    boxPathPositions = bluePathPositions
else
    boxPathPositions = blackPathPositions
end

```

```

boxPathIndex = 1
followBoxPath = true

```

```

state = "follow_box_path"
end

```

```

--

```

```

=====
=====

```

```

-- STATE: FOLLOW BOX TRANSPORT PATH
-- Move full box to collection area

```

```

--

```

```

=====
=====

```

```

if state == "follow_box_path" then

```

```

    if #boxPathPositions == 0 then
        state = "move_to_collection_area"
        return
    end

```

```

local goal = {
    boxPathPositions[3*(boxPathIndex-1)+1],

```

```

        boxPathPositions[3*(boxPathIndex-1)+2],
        boxPathPositions[3*(boxPathIndex-1)+3]
    }

    local ori = sim.getObjectOrientation(approachCollectionArea, -1)

    stepToward(goal, ori)
    solveIK()

    if reached(goal) then
        boxPathIndex = boxPathIndex + 1

        if boxPathIndex > (#boxPathPositions / 3) then
            boxPathIndex = 1
            followBoxPath = false

            -- ensure final stability
            sim.setObjectPosition(simTarget, -1, goal)
            currentTargetApproach = approachCollectionArea
            state = "move_to_collection_area"
        end
    end
end

--
=====
-- STATE: DEFECT RETURN PATH
-- Return robot after defect disposal
--
=====
=====

```



```

if state == "follow_defect_return_path" then

    if #defectReturnPathPositions == 0 then
        state = "move_up_after_place"
        return
    end

    local goal = {
        defectReturnPathPositions[3*(defectReturnPathIndex-1)+1],
        defectReturnPathPositions[3*(defectReturnPathIndex-1)+2],
        defectReturnPathPositions[3*(defectReturnPathIndex-1)+3]
    }

    local ori = sim.getObjectOrientation(approachPick, -1)

    stepToward(goal, ori)
    solveIK()

    if reached(goal) then
        defectReturnPathIndex = defectReturnPathIndex + 1

        if defectReturnPathIndex > (#defectReturnPathPositions / 3) then
            defectReturnPathIndex = 1
            followDefectReturnPath = false

            -- reset targets (important)
            currentTargetApproach = nil
            currentTargetPlace = nil

            -- go directly to waiting for next object

```

```

        state = "waiting"
    end
end
end
end

--
=====
=====

-- STATE: MOVE TO COLLECTION AREA
-- Transport full box toward storage location
--
=====
=====

if state=="move_to_collection_area" then
    local p,o = sim.getObjectPosition(currentTargetApproach,-1),
sim.getObjectOrientation(currentTargetApproach,-1)
    stepToward(p,o) solveIK()
    if reached(p) then state="place_box" end
end

--
=====
=====

-- STATE: PLACE BOX
-- Arrange boxes in collection grid
--
=====
=====

if state=="place_box" then

    if collectionCount >= 6 then collectionCount = 0 end

```

```

local base = sim.getObjectPosition(placeBox,-1)
local ori = sim.getObjectOrientation(placeBox,-1)

```

```

local col = collectionCount % 3
local row = math.floor(collectionCount / 3) % 2

```

```

local pos = {
    base[1] - col * 0.5,
    base[2] + row * 0.5,
    base[3]
}

```

```

stepToward(pos,ori) solveIK()
if reached(pos) then state="detach_box" end
end

```

```

--

```

```

=====
=====

```

```

-- STATE: DETACH BOX
-- Release full box and generate replacement box

```

```

--

```

```

=====
=====

```

```

if state=="detach_box" then
    local pos = sim.getObjectPosition(currentBoxHandle,-1)
    local ori = sim.getObjectOrientation(currentBoxHandle,-1)

    sim.setObjectParent(currentBoxHandle,-1,true)
    sim.setObjectPosition(currentBoxHandle,-1,pos)

```

```
sim.setObjectOrientation(currentBoxHandle,-1,ori)
```

```
collectionCount = collectionCount + 1
```

```
sim.setStringSignal("spawn_new_box", boxToHandle)
```

```
if boxToHandle=="red" then countRed=0
```

```
elseif boxToHandle=="blue" then countBlue=0
```

```
else countBlack=0 end
```

```
state="waiting"
```

```
end
```

```
--
```

```
=====
```

```
-- RESET COUNTERS AND SYSTEM VARIABLES
```

```
--
```

```
=====
```

```
local reset = sim.getInt32Signal("reset")
```

```
if reset == 1 then
```

```
    countRed, countBlue, countBlack, countYellow = 0,0,0,0
```

```
    collectionCount = 0
```

```
    sim.setInt32Signal("reset", 0)
```

```
end
```

```
--
```

```
=====
```

```
-- UPDATE UI COUNTERS
```

```
-- Send production statistics to interface
```

--

=====

```
sim.setInt32Signal("count_red", countRed)
sim.setInt32Signal("count_blue", countBlue)
sim.setInt32Signal("count_black", countBlack)
sim.setInt32Signal("count_yellow", countYellow)
sim.setInt32Signal("count_fullbox", collectionCount)
end
```

```
function sysCall_cleanup()
    simIK.eraseEnvironment(ikEnv)
end
```

Vision Sensor

```
--
=====
=====

-- VISION SENSOR: COLOUR DETECTION SCRIPT

--
=====
=====

-- This script performs object colour detection using a vision sensor.
-- The average RGB values are extracted from the sensor image, normalised, and
-- classified into one of four categories:

--
-- -> Red
-- -> Blue
-- -> Black
-- -> Yellow (default / defective object)

--
-- The detected colour is transmitted to the robot controller using
-- the signal "detectedColor".

--
=====
=====


--
=====
=====

-- INITIALISATION: sysCall_init()

--
=====
=====

-- Called once when the simulation starts.
-- Retrieves the handle of the attached vision sensor object.
```

```

--
=====

=====

function sysCall_init()
    sim = require('sim')
    vision_sensor = sim.getObject('..')
end

--
=====

=====

-- SENSING FUNCTION: sysCall_sensing()

--
=====

=====

-- Executed every simulation cycle.

--

-- Steps:
-- 1. Read vision sensor data
-- 2. Extract average RGB values
-- 3. Normalise colours
-- 4. Classify object colour
-- 5. Send detected result to robot controller

--
=====

=====

function sysCall_sensing()

    -- Read vision sensor output packet
    local result, packet1 = sim.readVisionSensor(vision_sensor)

```

-- Continue only if valid data exists

if result >= 0 and packet1 then

--

=====
=====

-- RGB EXTRACTION

-- Obtain average colour values from sensor packet

--

=====
=====

local avg_red = packet1[12] or 0

local avg_green = packet1[13] or 0

local avg_blue = packet1[14] or 0

--

=====
=====

-- NORMALISATION

-- Convert RGB values into ratios to reduce lighting effects

--

=====
=====

local sum = avg_red + avg_green + avg_blue

if sum == 0 then return end

local r = avg_red / sum

local g = avg_green / sum

local b = avg_blue / sum


```

--
=====
=====

-- COLOUR CLASSIFICATION
-- Default unknown objects to yellow category

--
=====
=====

local detectedColor = "yellow" -- DEFAULT EVERYTHING TO YELLOW

-- RED
if r > 0.5 and g < 0.3 then
    detectedColor = "red"

-- NAVY BLUE
elseif b > 0.45 and r < 0.25 and g < 0.25 then
    detectedColor = "blue"

-- BLACK
elseif r < 0.35 and g < 0.35 and b < 0.35 then
    detectedColor = "black"
end

--
=====
=====

-- DEBUG OUTPUT
-- Display detected RGB ratios and classification

--
=====
=====

```

```

print(string.format("R: %.2f G: %.2f B: %.2f -> %s", r, g, b, detectedColor))

--
=====
=====

-- SEND RESULT TO ROBOT CONTROLLER
-- Robot reads this signal for sorting decisions
--
=====
=====

if detectedColor ~= "unknown" then
    sim.setStringSignal("detectedColor", detectedColor)

end

end

end

--
=====
=====

-- CLEANUP: sysCall_cleanup()
--
=====
=====

-- Executed when simulation stops.
-- No cleanup operations are required for this script.
--
=====
=====

function sysCall_cleanup()

end

```

Proximity Sensor

```
--
=====
=====

-- PROXIMITY SENSOR: OBJECT DETECTION AND CONVEYOR CONTROL SCRIPT

--
=====
=====

-- This script controls the conveyor movement and detects objects using
-- a proximity sensor.

--

-- Functions:

-- 1. Detect incoming objects on the conveyor
-- 2. Stop conveyor when an object is detected
-- 3. Send object detection signals to robot controller
-- 4. Restart conveyor when area becomes clear

--

-- The robot receives:

-- detectedBox      -> indicates object presence
-- detectedObjectHandle -> handle of detected object

--
=====
=====

sim = require('sim')

--
=====
=====

-- GLOBAL VARIABLES

--
=====
=====
```

```
local sensorHandle
```

```
local conveyorHandle
```

```
-- Conveyor operating speed
```

```
local targetVelocity = 0.05
```

```
--
```

```
=====
```

```
-- INITIALISATION: sysCall_init()
```

```
--
```

```
=====
```

```
-- Executed once at simulation start.
```

```
-- Retrieves handles for proximity sensor and conveyor.
```

```
--
```

```
=====
```

```
function sysCall_init()
```

```
-- Get attached proximity sensor handle
```

```
sensorHandle = sim.getObject("../proximitySensor")
```

```
-- Get conveyor object handle
```

```
conveyorHandle = sim.getObject("/conveyor")
```

```
end
```

```
--
```

```
=====
```

```
-- SENSING FUNCTION: sysCall_sensing()
```

```

--
=====
=====

-- Executed every simulation cycle.

--

-- Workflow:

-- 1. Check system status
-- 2. Read proximity sensor
-- 3. Stop conveyor if object detected
-- 4. Send detection signals to robot
-- 5. Restart conveyor when object leaves

--
=====
=====

```

```

function sysCall_sensing()

```

```

--
=====
=====

-- SYSTEM SAFETY CHECK
-- Stop conveyor completely if system disabled

--
=====
=====

```

```

if sim.getInt32Signal("system_enable") == 0 then
    sim.writeCustomTableData(conveyorHandle, "__ctrl__", {vel = 0})
    return
end

```

```

--
=====
=====

-- OBJECT DETECTION
-- Read proximity sensor measurements

--
=====
=====

local result, distance, point, detectedObject = sim.readProximitySensor(sensorHandle)

--
=====
=====

-- OBJECT FOUND
-- Stop conveyor and notify robot controller

--
=====
=====

if result == 1 then

    -- Stop conveyor motion
    sim.writeCustomTableData(conveyorHandle, "__ctrl__", {vel = 0})

    -- Send detection status
    sim.setInt32Signal("detectedBox", 1)

    -- Send detected object handle
    sim.setInt32Signal("detectedObjectHandle", detectedObject)

```

```

--
=====
=====

-- NO OBJECT DETECTED
-- Resume conveyor operation

--
=====
=====

else

    -- Run conveyor at preset speed
    sim.writeCustomTableData(conveyorHandle, "__ctrl__", {vel = targetVelocity})

    -- Reset detection signal
    sim.setInt32Signal("detectedBox", 0)

    -- Clear object handle
    sim.setInt32Signal("detectedObjectHandle", -1)
end
end

--
=====
=====

-- CLEANUP: sysCall_cleanup()

--
=====
=====

-- Executed when simulation ends.
-- Resets detection signals to avoid stale data.

```

--

=====

function sysCall_cleanup()

-- Clear object detection flag

sim.setInt32Signal("detectedBox", 0)

-- Clear stored object handle

sim.setInt32Signal("detectedObjectHandle", -1)

end

UI Script

```
sim = require('sim')
```

```
simUI = require('simUI')
```

```
--
```

```
=====
```

```
-- PRODUCTION CELL CONTROL PANEL SCRIPT
```

```
--
```

```
=====
```

```
-- This script manages the supervisory layer of the production cell.
```

```
--
```

```
-- Functions included:
```

```
--
```

```
-- 1. Production control UI
```

```
-- - Start / Stop system
```

```
-- - Manual or automatic operation
```

```
-- - Normal and collaborative modes
```

```
-- - Robot speed control
```

```
--
```

```
-- 2. Object generation
```

```
-- - Manual spawning of coloured phones
```

```
-- - Automatic weighted spawning
```

```
-- - Automatic replacement of full storage boxes
```

```
--
```

```
-- 3. Safety monitoring
```

```
-- - Reads safety scanner distance
```

```
-- - Updates SAFE / WARNING / DANGER status
```

```
--
```

```
-- 4. Runtime monitoring
```

```
-- - Displays live object counts
```

- - Tracks full collection boxes
- - Resets production statistics
-
- Communication with the robot controller is achieved using CoppeliaSim
- signals. The robot script reads these signals and performs pick, place,
- sorting and box handling operations.

--

```
=====
=====
```

function sysCall_init()

--

```
=====
=====
```

-- 1. SAFETY SETTINGS

--

```
=====
=====
```

-- Distance thresholds used for human detection and safety indication:

--

-- dangerDist -> robot stop zone

-- warningDist -> reduced speed zone

-- safe zone -> normal operation

--

```
=====
=====
```

dangerDist = 0.5

warningDist = 2.0

--

```
=====
=====
```

-- 2. SYSTEM MODES

--

```
=====
=====
```

- systemOn : enables or disables whole production cell
- autoMode : enables automatic spawning operation
- Default startup mode = manual operation

--

```
=====
=====
```

systemOn = true

autoMode = false

--

```
=====
=====
```

-- 3. INITIAL SIGNAL VALUES

--

```
=====
=====
```

- Signals provide communication between UI, spawner and robot controller.

--

- ui_speed_factor -> robot speed scaling
- mode -> normal / collaborative operation
- safety_distance -> scanner reading

--

```
=====
=====
```

sim.setFloatSignal("ui_speed_factor", 1.0) -- default 100%

sim.setInt32Signal("mode", 0) -- 0 = NORMAL, 1 = COLLAB

sim.setFloatSignal("safety_distance", 10) -- default SAFE

```

--
=====
=====

-- 4. OBJECT DEFINITIONS

--
=====
=====

-- Maps user colour requests to phone templates used for spawning.

--
-- Example:
-- red -> redPhone

--
=====
=====

```

```

objectMap = {
    red = "redPhone",
    blue = "bluePhone",
    black = "blackPhone",
    yellow = "yellowPhone"
}

```

```

--
=====
=====

-- 5. STORAGE BOX DEFINITIONS

--
=====
=====

```

```

-- Template boxes are cloned whenever full containers are removed from
-- the sorting station.

```

```
--
=====
=====
```

```
boxTemplateMap = {
    red = "redPhoneBox",
    blue = "bluePhoneBox",
    black = "blackPhoneBox"
}
```

```
--
=====
=====
```

```
-- 6. BOX REPLACEMENT POSITIONS
```

```
--
=====
=====
```

```
-- Stores original coordinates of storage boxes so replacements appear
-- at correct locations.
```

```
--
=====
=====
```

```
boxSpawnPos = {
    red = sim.getObjectPosition(sim.getObject("/redPhoneBox"), -1),
    blue = sim.getObjectPosition(sim.getObject("/bluePhoneBox"), -1),
    black = sim.getObjectPosition(sim.getObject("/blackPhoneBox"), -1)
}
```

```
--
=====
=====
```

```
-- 7. PHONE SPAWN LOCATION
```

```

--
=====
=====

-- Position used when placing new phones on conveyor input.

--
=====
=====

spawnPos = {-2.698, -0.55, 0.710}

--
=====
=====

-- 8. AUTOMATIC SPAWN SETTINGS

--
=====
=====

-- interval    -> time between auto spawned objects in seconds
-- lastSpawnTime -> stores previous spawn instant

--
=====
=====

interval = 2
lastSpawnTime = sim.getSimulationTime()

--
=====
=====

-- 9. INITIALISE COMMUNICATION SIGNALS

--
=====
=====

-- Reset all counters and prepare robot communication.

```

```

--
=====

sim.setInt32Signal("system_enable", 1)
sim.setInt32Signal("auto_mode", 0)
sim.setStringSignal("spawn_color", "")
sim.setInt32Signal("spawned_case_handle", 0)

sim.setInt32Signal("count_red", 0)
sim.setInt32Signal("count_blue", 0)
sim.setInt32Signal("count_black", 0)
sim.setInt32Signal("count_yellow", 0)
sim.setInt32Signal("count_fullbox", 0)

--
=====
=====

-- 10. USER INTERFACE CREATION

--
=====
=====

-- Floating control panel containing:

--
-- - System control
-- - Manual spawning
-- - Safety indicator
-- - Speed control
-- - Colour probability settings
-- - Production statistics

--
=====
=====

```

```

local xml = [[
<ui title="Production Control Panel" closeable="false" on-top="true" layout="vbox">

<!-- TOP ROW -->
<group layout="hbox">
  <group layout="vbox">

    <label text="SYSTEM CONTROL"/>
    <button text="Start / Stop System" on-click="toggleSystem"/>
    <button text="Toggle Auto Mode" on-click="toggleAuto"/>

    <label text=" "/>

    <label text="MODE"/>
    <button text="Normal Mode" on-click="modeNormal"/>
    <button text="Collaborative Mode" on-click="modeCobot"/>

    <label text=" "/>

    <label text="ROBOT SPEED"/>
    <edit id="301" value="100"/>
    <button text="Apply Speed" on-click="applySpeed"/>

  </group>

  <group layout="vbox">

    <label text="MANUAL SPAWN"/>
    <button text="Red" on-click="spawnRed"/>
    <button text="Blue" on-click="spawnBlue"/>

```



```
<button text="Black" on-click="spawnBlack"/>
<button text="Yellow" on-click="spawnYellow"/>
```

```
<label text=" "/>
```

```
<label text="SAFETY STATUS"/>
<label text="SAFE" id="400" style="* {font-size: 22px; color: green;}" />
```

```
<label text=" "/>
```

```
<button text="RESET SYSTEM" on-click="resetSystem"/>
```

```
</group>
```

```
</group>
```

```
<!-- BOTTOM ROW -->
```

```
<group layout="hbox">
```

```
<group layout="vbox">
```

```
<label text="SET COLOR (%)" />
<label text="Red" /><edit id="201" value="33" />
<label text="Blue" /><edit id="202" value="33" />
<label text="Black" /><edit id="203" value="33" />
<label text="Yellow" /><edit id="204" value="1" />
```

```
</group>
```

```
<group layout="vbox">
```

```
<label text="LIVE COUNTS" />
<label text="Red: 0" id="101" />
<label text="Blue: 0" id="102" />
<label text="Black: 0" id="103" />
```

```

        <label text="Yellow: 0" id="104"/>
        <label text=" "/>
        <label text="Full Boxes: 0" id="105"/>
    </group>

</group>

</ui>
]]

ui = simUI.create(xml)

print("UI initialized successfully")
end

--
=====
=====
-- MODE CONTROL FUNCTIONS
--
=====
=====
-- modeNormal():
--   Restores full speed operation.
--
-- modeCobot():
--   Enables collaborative mode and limits speed.
--
=====
=====

function modeNormal()

```

```

sim.setInt32Signal("mode", 0)

simUI.setEditValue(ui, 301, "100")
sim.setFloatSignal("ui_speed_factor", 1.0)

print("Mode: NORMAL")
end

function modeCobot()
    sim.setInt32Signal("mode", 1)

    -- force safe speed visually + internally
    simUI.setEditValue(ui, 301, "30")
    sim.setFloatSignal("ui_speed_factor", 0.3)

    print("Mode: COLLABORATIVE")
end

--
=====
=====

-- COLOUR DISTRIBUTION LOGIC
--
=====
=====

-- Generates weighted random colours using percentages entered in UI.
--
-- Example:
-- Red = 50
-- Blue = 20
-- Black = 20
-- Yellow = 10

```

```

--
-- Red phones will appear more frequently.
--
=====
=====

function getWeightedColor()

    local r = tonumber(simUI.getEditValue(ui, 201)) or 0
    local g = tonumber(simUI.getEditValue(ui, 202)) or 0
    local b = tonumber(simUI.getEditValue(ui, 203)) or 0
    local y = tonumber(simUI.getEditValue(ui, 204)) or 0

    local total = r + g + b + y
    if total == 0 then r,g,b,y,total = 33,33,33,1,100 end

    local rand = math.random() * total

    if rand < r then return "red"
    elseif rand < r + g then return "blue"
    elseif rand < r + g + b then return "black"
    else return "yellow" end
end

--
=====
=====

-- USER BUTTON CALLBACKS
--
=====
=====

-- Handles:

```

```

-- - System ON/OFF
-- - Auto mode switching
-- - Manual phone spawning
--
=====
=====

function toggleSystem()
    systemOn = not systemOn
    sim.setInt32Signal("system_enable", systemOn and 1 or 0)
    print(systemOn and "System ON" or "System OFF")
end

function toggleAuto()
    autoMode = not autoMode
    sim.setInt32Signal("auto_mode", autoMode and 1 or 0)
    print(autoMode and "Auto Mode ON" or "Manual Mode ON")
end

--
=====
=====

-- MANUAL SPAWN CALLBACKS
--
=====
=====

-- These functions are triggered when the user presses the manual spawn
-- buttons from the UI panel. The selected colour is written to the
-- "spawn_color" signal which is later read inside sysCall_sensing().
--
=====
=====

```

```

function spawnRed() sim.setStringSignal("spawn_color", "red") end
function spawnBlue() sim.setStringSignal("spawn_color", "blue") end
function spawnBlack() sim.setStringSignal("spawn_color", "black") end
function spawnYellow() sim.setStringSignal("spawn_color", "yellow") end

```

```

--
=====
=====

-- SPEED CONTROL: applySpeed()

--
=====
=====

-- Reads the speed percentage entered in the UI.
-- The value is limited between 0?100%.
-- The final speed scaling factor is stored in "ui_speed_factor"
-- and later used by the robot motion script.

--
=====
=====

```

```

function applySpeed()
    local val = tonumber(simUI.getEditValue(ui, 301)) or 100
    if val < 0 then val = 0 end
    if val > 100 then val = 100 end

    local speedFactor = val / 100
    sim.setFloatSignal("ui_speed_factor", speedFactor)

    print("Speed set to:", speedFactor)
end

```

```

--
=====
=====

-- SYSTEM RESET: resetSystem()

--
=====
=====

-- Resets all production counters including:
-- Red objects
-- Blue objects
-- Black objects
-- Defective objects
-- Full collection boxes

--

-- A reset signal is also sent so the robot script can clear its
-- internal counters and restart the cycle.

--
=====
=====

function resetSystem()
    sim.setInt32Signal("count_red", 0)
    sim.setInt32Signal("count_blue", 0)
    sim.setInt32Signal("count_black", 0)
    sim.setInt32Signal("count_yellow", 0)
    sim.setInt32Signal("count_fullbox", 0)

    sim.setInt32Signal("reset", 1)

    print("System Reset Triggered")
end

```

```

--
=====
=====

-- OBJECT SPAWNING FUNCTIONS

--
=====
=====

-- spawnObject():
--   Creates phone instances at conveyor location.
--
-- spawnNewBox():
--   Replaces full storage boxes after collection.
--
=====
=====

function spawnObject(name)
  local templateHandle = sim.getObject("/" .. name)
  if templateHandle ~= -1 then
    local obj = sim.copyPasteObjects({templateHandle}, 0)[1]
    sim.setObjectPosition(obj, -1, spawnPos)
    sim.setInt32Signal("spawned_case_handle", obj)
  end
end

function spawnNewBox(color)

  local templateName = boxTemplateMap[color]
  if not templateName then return end

  local templateHandle = sim.getObject("/" .. templateName)

```



```

if templateHandle ~= -1 then
    local newBox = sim.copyPasteObjects({templateHandle}, 0)[1]
    sim.setObjectPosition(newBox, -1, boxSpawnPos[color])

    if color == "red" then
        sim.setInt32Signal("new_red_box", newBox)
    elseif color == "blue" then
        sim.setInt32Signal("new_blue_box", newBox)
    elseif color == "black" then
        sim.setInt32Signal("new_black_box", newBox)
    end
end
end
end

--
=====
=====
-- SAFETY DISPLAY UPDATE: updateSafetyUI()
--
=====
=====
-- Reads the safety distance signal generated by the monitoring system.
--
-- Safety states:
-- SAFE    -> Human/object outside warning zone
-- WARNING -> Human/object inside warning zone
-- DANGER  -> Human/object inside emergency stop zone
--
-- The UI colour and text are updated accordingly.
--
=====
=====

```

```

function updateSafetyUI()

    local dist = sim.getFloatSignal("safety_distance")

    if not dist then
        simUI.setLabelText(ui, 400, "NO DATA")
        return
    end

    if dist < dangerDist then
        simUI.setLabelText(ui, 400, "DANGER")
        simUI.setStyleSheet(ui, 400, "* {color: red; font-size: 20px;}")

    elseif dist < warningDist then
        simUI.setLabelText(ui, 400, "WARNING")
        simUI.setStyleSheet(ui, 400, "* {color: orange; font-size: 20px;}")

    else
        simUI.setLabelText(ui, 400, "SAFE")
        simUI.setStyleSheet(ui, 400, "* {color: green; font-size: 20px;}")
    end
end

--
=====
=====
-- MAIN SENSING LOOP
--
=====
=====
-- Executed every simulation step.

```

```
--
-- Sequence:
--
-- 1. Update safety status
-- 2. Replace removed boxes
-- 3. Update live counters
-- 4. Check system state
-- 5. Handle manual spawn requests
-- 6. Execute automatic spawning
--
-- The loop acts as the supervisory controller of the production cell.
--
=====
```

```
function sysCall_sensing()
```

```
    if ui then updateSafetyUI() end
```

```
    local newBoxReq = sim.getStringSignal("spawn_new_box")
```

```
    if newBoxReq and newBoxReq ~= "" then
```

```
        spawnNewBox(newBoxReq)
```

```
        sim.setStringSignal("spawn_new_box", "")
```

```
    end
```

```
    if ui then
```

```
        simUI.setLabelText(ui,101,"Red: "..(sim.getInt32Signal("count_red") or 0))
```

```
        simUI.setLabelText(ui,102,"Blue: "..(sim.getInt32Signal("count_blue") or 0))
```

```
        simUI.setLabelText(ui,103,"Black: "..(sim.getInt32Signal("count_black") or 0))
```

```
        simUI.setLabelText(ui,104,"Yellow: "..(sim.getInt32Signal("count_yellow") or 0))
```

```
        simUI.setLabelText(ui,105,"Full Boxes: "..(sim.getInt32Signal("count_fullbox") or 0))
```

```

end

if sim.getInt32Signal("system_enable") == 0 then return end

if (sim.getInt32Signal("spawned_case_handle") or 0) ~= 0 then return end

local req = sim.getStringSignal("spawn_color")
if req and req ~= "" then
    spawnObject(objectMap[req])
    sim.setStringSignal("spawn_color", "")
    return
end

if sim.getInt32Signal("auto_mode") == 1 then
    if sim.getSimulationTime() - lastSpawnTime >= interval then
        spawnObject(objectMap[getWeightedColor()])
        lastSpawnTime = sim.getSimulationTime()
    end
end
end
end

--
=====
=====
-- CLEANUP
--
=====
=====
-- Removes UI resources when simulation stops.
-- Prevents unused windows remaining after execution.

```

--

=====

```
function sysCall_cleanup()
    if ui then simUI.destroy(ui) end
end
```